IT შეხვედრის კითხვები და პასუხები

> **ვისთვის:** ჩემთვის და სალომესთვის — NAPR-ის IT სამსახურთან შეხვედრისთვის მოსამზადებლად. **მდგომარეობა:** 2026-10-22. ფაქტები ემყარება კოდის აუდიტს (01 — Handoff). ცნებების ახსნა იხ. 02 — სახელმძღვანელოში. **სტატუსის ნიშნები:** ✅ დადასტურებული · 🗓️ დაგეგმილი · 💡 რეკომენდებული · ❓ IT-ის დასადასტურებელი · ⚠️ შეუსაბამობა.

როგორ გამოვიყენოთ

ყოველ კითხვას აქვს:

- 🗣️ **მოკლე პასუხი** — 2–4 წინადადება, შეხვედრაზე ხმამაღლა სათქმელი;
- 🔧 **ტექნიკური ახსნა** — თუ ჩაგეკითხნენ;
- 💬 **მარტივად რომ გავიგოთ** — ჩემთვის, რომ ნამდვილად მესმოდეს;
- 📌 **ჩვენი პროექტიდან** — კონკრეტული ფაქტი, ფაილი ან ციფრი;
- ❓ **გაურკვეველია** — რა არ ვიცით (გულწრფელად);
- ➡️ **რა ვკითხოთ IT-ს** — როგორ გადავიყვანოთ საუბარი შემდეგ ნაბიჯზე.

სირთულე: 🟢 საბაზისო · 🟡 საშუალო · 🔴 რთული ან წინააღმდეგობა.

სამი ოქროს წესი:

1. რაც არ ვიცით, ვამბობთ: „ეს არ ვიცით, გავარკვევთ და მოგწერთ“. რიცხვს არ ვიგონებთ.
2. პროტოტიპს ისე ვაღწერთ, როგორც არის: „ლოკალური პროტოტიპი რეალურ მონაცემზე“, და არა „მზა სისტემა“.
3. IT-ის სტანდარტს პატივს ვცემთ: „თქვენი სტანდარტი გვირჩევნია“. სადაც კომპრომისი შეუძლებელია, ვხსნით, რატომ.

სარჩევი

- **ა. ზოგადი და მენეჯმენტი** — Q1–Q8
- **ბ. სერვერი და ინფრასტრუქტურა** — Q9–Q20
- **გ. მონაცემთა ბაზა** — Q21–Q31
- **დ. ინტეგრაცია და მონაცემი** — Q32–Q43
- **ე. უსაფრთხოება** — Q44–Q54
- **ვ. მიგრაცია, ოპერაციები, მხარდაჭერა** — Q55–Q62

ა. ზოგადი და მენეჯმენტი

Q1. 🟢 რა არის ეს სისტემა და რატომ გჭირდებათ?

🗣️ **მოკლე პასუხი:** „ეს ხარისხის კონტროლის ერთიანი ვებ-პლატფორმაა. ხუთ ცალკე ინსტრუმენტს ცვლის: Google ცხრილს, მის სკრიპტებს, განაწილების დესკტოპ-პროგრამას, სტატისტიკის პანელს და Pre-QC-ის შედეგებს. ≈ 300 თანამშრომელი ერთ ადგილას იმუშავებს, თავისი როლით. ყოველი ნაბიჯი ისტორიაში ინახება, და წესებს სისტემა იცავს.“

🔧 **ტექნიკური ახსნა:** FastAPI-ზე (Python) აგებული მონოლითური ვებ-აპლიკაციაა, სერვერზე აწყობილი HTML-ით (Jinja2 + HTMX), რელაციური ბაზით (SQLAlchemy; პროტოტიპში SQLite, საწარმოოში PostgreSQL), RBAC-ით და საკუთრების ფილტრებით. სამუშაო პროცესი state machine-ია: 7 მდგომარეობა, append-only ისტორია. წყარო სისტემებს მხოლოდ კითხულობს.

💬 **მარტივად რომ გავიგოთ:** დღეს ოქმი რამდენიმე პროგრამასა და ცხრილს შორის „დადის“, და ყველაფერს ადამიანები ამოწმებენ. ახალ სისტემაში ოქმი ერთ ადგილას ცხოვრობს, ზუსტად ჩანს, ვისთანაა, და არასწორი ნაბიჯი შეუძლებელია.

📌 **ჩვენი პროექტიდან:** ✅ ლოკალური პროტოტიპი v0.2.0; 44 159 რეალური შემთხვევა; 605 მომხმარებელი; 7 როლი; 57 ტესტი.

❓ **გაურკვეველია:** —

➡️ **რა ვკითხოთ IT-ს:** „ჩვენი მოთხოვნები დოკუმენტში გვაქვს (04). რომელი ნაწილის განხილვით დავიწყოთ: სერვერით თუ მონაცემის წვდომით?“

Q2. 🟢 რატომ არ შეიძლება Google ცხრილების გაგრძელება? ხომ მუშაობს?

🗣️ **მოკლე პასუხი:** „მუშაობს, მაგრამ ზღვარზეა. ცხრილი 30 ათას მწკრივს გადასცდა, მისი წაკითხვა სკრიპტით ~85 წამს გრძელდება და ხანდახან ვარდება. ისტორია არ ინახება, რადგან სტატუსი ზედ იწერება. ხილვადობას ცხრილის გაზიარება მართავს და არა როლები. და ბიზნეს-წესებს ადამიანები იცავენ და არა სისტემა.“

🔧 **ტექნიკური ახსნა:**

- Google Sheets ელცხრილია და არა ტრანზაქციული ბაზა: არ აქვს ბლოკირება მწკრივის დონეზე, შეზღუდვები (constraints) და ისტორია ველის დონეზე;
- Apps Script-ს შესრულების ლიმიტი აქვს: 6 წუთი ჩვეულებრივ ანგარიშზე;
- ერთდროული ჩაწერა `LockService`-ით იმართება;
- „სარკისებრი“ ჩანართები ფორმულებით იწყობა.

💬 **მარტივად რომ გავიგოთ:** ცხრილი რვეულია, რომელშიც ბევრი წერს ერთდროულად. წერა შეიძლება, მაგრამ ვერავინ გეტყვის, ვინ წაშალა წინა ჩანაწერი, და რვეული ვერ

შეგაჩერებს, თუ არასწორად წერ.

📌 **ჩვენი პროექტიდან:** ✅ `NAPR_Distributor/README.md`-ში აღწერილია Apps Script-ის ჩავარდნები („Service Spreadsheets failed…“), 3-ჯერადი ხელახალი ცდა და ნაწილობრივი ჩაწერის აღდგენა. ✅ ძველი ცხრილიდან გადმოტანისას 379 დუბლიკატი აღმოჩნდა.

❓ **გაურკვეველია:** —

➡️ **რა ვკითხოთ IT-ს:** „Google Workspace სააგენტოს ოფიციალური ინსტრუმენტია, თუ ცხრილები პირად ანგარიშებზეა? ეს Q46-საც ეხება.“

Q3. 🔴 რატომ ააშენეთ თავად? რატომ არ გამოიყენეთ მზა პროდუქტი ან არსებული სისტემის მოდული?

🗣️ **მოკლე პასუხი:** „ჩვენი სამუშაო პროცესი სპეციფიკურია: „არ ვეთანხმები“ მენეჯერის დადასტურებით, QC მენეჯერის სამი გადაწყვეტილება, CC პრინციპი, ბლოკური განაწილება პროცენტებით. ამას მზა ინსტრუმენტი ღრმა მორგების გარეშე ვერ ასრულებს. თუ სააგენტოს სისტემაში ასეთი მოდული არსებობს, სიამოვნებით განვიხილავთ. ჩვენი წესები და მონაცემის მოდელი დოკუმენტირებულია და ნებისმიერ პლატფორმაზე გადაიტანება.“

🔧 **ტექნიკური ახსნა:** ზოგადი workflow ან ticketing სისტემები (Jira და მისთ.) იძლევა მდგომარეობებსა და როლებს, მაგრამ არ იძლევა:

- **(ა)** REGFIELDALL-ზე „პირველი ოქმის“ კანდიდატების გამოთვლას;
- **(ბ)** ბლოკურ, ფენოვან, პროცენტულ შერჩევას;
- **(გ)** ოქმის რამდენიმე ამზომველსა და რამდენიმე მენეჯერზე მიკუთვნებას;
- **(დ)** სტატისტიკას ჩვენი მეტრიკების ლექსიკონით.

💬 **მარტივად რომ გავიგოთ:** მზა „კოსტიუმი“ არსებობს, მაგრამ ჩვენს ზომაზე ბევრი გადაკერება სჭირდება. გადაკერება კი ზოგჯერ ახლის შეკერვაზე ძვირია.

📌 **ჩვენი პროექტიდან:** ✅ წესები შეთანხმებულია 2026-10-19/20-ზე (`docs/OPEN_QUESTIONS.md`, „მიღებული გადაწყვეტილებები“). ✅ ბირთვული ალგორითმები ძველი პროგრამებიდან გადმოვიდა.

❓ **გაურკვეველია:** ❓ აქვს თუ არა სააგენტოს შიდა BPM ან workflow პლატფორმა.

➡️ **რა ვკითხოთ IT-ს:** „გაქვთ შიდა სტანდარტული პლატფორმა ასეთი აპლიკაციებისთვის? თუ კი, რა მოთხოვნები აქვს?“

Q4. 🔴 ვინ იქნება ადმინისტრატორი და ვინ დაუჭერს მხარს?

🗣️ **მოკლე პასუხი:** „ბიზნეს-ადმინისტრირებას — მომხმარებლებს, როლებს, გუნდებს, კოდებს, განაწილებას — ჩვენ ვაკეთებთ, ორი ადმინი: მე და სალომე დავითაშვილი.

ინფრასტრუქტურას (სერვერი, ბაზა, ბექაპი, ქსელი) თქვენთან ვთავაზობთ. კოდის განვითარება და შეცდომების გასწორება ჩვენზეა. ეს გაყოფა წერილობით ჩავწეროთ.“

🔧 **ტექნიკური ახსნა:** RACI-ის შეთავაზება:

სფერო	ვინ
OS, პაჩები, ქსელი, firewall, სერტიფიკატი	IT
PostgreSQL-ის ექსპლუატაცია, ბექაპი, აღდგენა	IT (DBA)
აპლიკაციის განახლება	ერთად, პროცედურით
აპლიკაციის ადმინისტრირება	QC (ლუკა, სალომე)
ინციდენტი	პირველ რიგში IT (ინფრასტრუქტურა), მეორეში ჩვენ (აპლიკაცია)

💬 **მარტივად რომ გავიგოთ:** IT „შენობას“ უვლის: შუქი, წყალი, დაცვა. ჩვენ „მაღაზიას“: თაროები, ფასები, თანამშრომლები.

📌 **ჩვენი პროექტიდან:** ✅ ადმინის როლი მხოლოდ ორ ადამიანს აქვს. ✅ ადმინისტრირება → მომხმარებლები, გუნდები, კოდები, პარამეტრები, მეილები, აუდიტი.

❓ **გაურკვეველია:** ❓ IT-ის ინციდენტების სამსახურის (helpdesk) პროცესი; მორიგეობა არასამუშაო საათებში.

➡️ **რა ვკითხოთ IT-ს:** „ახალი აპლიკაციისთვის რა სტანდარტული ოპერაციული შეთანხმება (SLA / RACI) გაქვთ?“

Q5. 🟢 IT-სგან რა გჭირდებათ კონკრეტულად?

🗣️ **მოკლე პასუხი:** „პილოტისთვის ხუთი რამ: ერთი VM, PostgreSQL, დომენი HTTPS-ით, Google OAuth client და SMTP relay. შემდეგ ეტაპზე წყაროებზე read-only წვდომა: REGFIELDALL, სპორადულის სია, ოქმის დეტალები. სრული სია მიზეზებით და ალტერნატივებით დოკუმენტ 04-შია.“

🔧 **ტექნიკური ახსნა:** იხ. 04 — IT ინფრასტრუქტურის მოთხოვნები: R1–R26, პრიორიტეტებითა და ფაზებით.

💬 **მარტივად რომ გავიგოთ:** ჯერ „ოთახი“ (სერვერი), „კარადა“ (ბაზა), „მისამართი“ (დომენი), „შესასვლელი“ (Google) და „ფოსტა“ (SMTP). მერე „გასაღებები“ მონაცემის წყაროებზე.

📌 **ჩვენი პროექტიდან:** ✅ კოდში Google OAuth და SMTP უკვე ჩაშენებულია. ჩასართავად მხოლოდ კონფიგურაცია სჭირდება (`QCP_GOOGLE_*` , `QCP_SMTP_*`).

❓ **გაურკვეველია:** —

➡️ **რა ვკითხოთ IT-ს:** „ამ სიიდან რა არის თქვენთვის სტანდარტული, და რა მოითხოვს ცალკე დამტკიცებას?“

Q6. 🟢 რა არის მზად და რა — არა?

🗣️ **მოკლე პასუხი:** „მზადაა ფუნქციონალი: სრული სამუშაო პროცესი, განაწილება, სტატისტიკა, როლები, ისტორია, რეალურ მონაცემზე და ტესტებით. მზად არ არის საწარმოო გარემო. ჩვენი მხრიდან დარჩა PostgreSQL-ზე გადასვლა და ტესტირება, მიგრაციების ინსტრუმენტი, ფონური ამოცანები და რამდენიმე უსაფრთხოების გამკაცრება. Pre-QC ჯერ დემოა.“

🔧 **ტექნიკური ახსნა:** იხ. Handoff §7 და §8.3 (G1–G18). მაგალითები:

- `psycopg` არ აყენია;
- `create_all` Alembic-ის ნაცვლად;
- გადახედვა პროცესის მეხსიერებაში;
- სინქრონული SMTP;
- დემო შესვლა ნაგულისხმევად ჩართულია.

💬 **მარტივად რომ გავიგოთ:** მანქანა აწყობილია და ეზოში დადის, მაგრამ გზაზე გასასვლელად ნომერი, დაზღვევა და ტექდათვალიერება სჭირდება.

📌 **ჩვენი პროექტიდან:** ✅ 57 ტესტი გადის; ✅ 0 ელფოსტა ბაზაში (Google-ით შესვლის წინაპირობა).

❓ **გაურკვეველია:** ❓ სპორადულის ფილტრები (Q11 ა–დ), კოორდინატორისა და სამსახურის უფროსის როლები, Pre-QC-ის შეყვანის ფორმატი.

➡️ **რა ვკითხოთ IT-ს:** „საწარმოომდე თქვენი შემოწმების სია გაქვთ (security review, checklist)? ჩვენს სიას შევადარებთ.“

Q7. 🟡 როდის შეიძლება გაშვება?

🗣️ **მოკლე პასუხი:** „ეს ორ რამეზეა დამოკიდებული: როდის გვექნება სერვერი და წვდომები, და ჩვენს დარჩენილ ტექნიკურ სამუშაოზე. ეს უკანასკნელი შეგვიძლია ინფრასტრუქტურის მომზადების პარალელურად ვაკეთოთ. ვთავაზობთ ეტაპობრივად: ჯერ პილოტი მცირე ჯგუფზე, დღევანდელი მონაცემის გზით; შემდეგ წყაროების ჩართვა.“

🔧 **ტექნიკური ახსნა:** ფაზები (Handoff §7.4):

- **0** — მომზადება;
- **1** — პილოტი: VM + PostgreSQL + დომენი + OAuth;
- **2** — წყაროები;
- **3** — სრული გაშვება + SMTP + cutover;

- **4** — Pre-QC.

💬 **მარტივად რომ გავიგოთ:** ერთ დღეში ყველაფერს არ ვრთავთ. ჯერ პატარა ჯგუფი ცდის, მერე ყველა.

📌 **ჩვენი პროექტიდან:** —

❓ **გაურკვეველია:** ❓ ზუსტ ვადას ვერ ვასახელებ, სანამ IT-ის ვადები ცნობილი არ არის. 💡 შეხვედრაზე თარიღს არ ვპირდებით, ჯერ IT-ის ვადებს ვითხოვთ.

➡️ **რა ვკითხოთ IT-ს:** „VM-ის, ბაზისა და დომენის გამოყოფას ჩვეულებრივ რამდენი დრო სჭირდება?“

Q8. 🔴 თუ თქვენ (ავტორები) წახვალთ, ვინ გააგრძელებს?

🗣️ **მოკლე პასუხი:** „სამართლიანი კითხვაა, და ეს რეალური რისკია. ამიტომ ვიყენებთ სტანდარტულ ტექნოლოგიებს (Python, FastAPI, PostgreSQL), ვწერთ ტესტებსა და დოკუმენტაციას. ორი ადამიანი ვიცით სისტემა. გთავაზობთ, რომ თქვენი ერთი დეველოპერი ან ადმინისტრატორი თავიდანვე ჩაერთოს.“

🔧 **ტექნიკური ახსნა:** knowledge transfer-ის ზომები:

- ✅ ქართული დოკუმენტაცია (ARCHITECTURE, IT_REQUEST, DEMO, OPEN_QUESTIONS + ეს handoff);
- ✅ კოდის კომენტარები და ბიზნეს-წესები ერთ მოდულში (`workflow.py` , `domain.py`);
- ✅ 57 ტესტი;
- ⚠️ ვერსიის კონტროლი ჯერ არ არის (G17) — 💡 git-ის რეპოზიტორია სააგენტოს სერვერზე.

💬 **მარტივად რომ გავიგოთ:** სახლს „ნახაზები“ უნდა ჰქონდეს, რომ სხვა ოსტატმაც გაიგოს, სად გადის მილები.

📌 **ჩვენი პროექტიდან:** ✅ `docs/handoff/` — 6 დოკუმენტი.

❓ **გაურკვეველია:** ❓ IT-ს აქვს თუ არა Python-ის დეველოპერი.

➡️ **რა ვკითხოთ IT-ს:** „კოდის საცავი (GitLab ან სხვა) გაქვთ? შეგვიძლია კოდი იქ განვათავსოთ, რომ თქვენც ხედავდეთ?“

ბ. სერვერი და ინფრასტრუქტურა

Q9. 🟡 რატომ გჭირდებათ ცალკე სერვერი? ჩვენს არსებულ ვებ-სერვერზე ვერ დადგება?

🗣️ **მოკლე პასუხი:** „ცალკე ფიზიკური სერვერი არ გვჭირდება. ერთი VM საკმარისია, ან ადგილი თქვენს არსებულ აპლიკაციის სერვერზე, თუ Python-ის აპლიკაციის გაშვება იქ შეიძლება. მთავარია, ჩვენი პროცესი იზოლირებული იყოს და ბაზა ცალკე ჰქონდეს.“

🔧 **ტექნიკური ახსნა:** აპლიკაციას სჭირდება:

- Python ≥ 3.12 runtime;
- მუდმივი სერვისი (uvicorn);
- reverse proxy;
- ფონური ამოცანები;
- ფაილების საცავი;
- PostgreSQL-თან კავშირი.

ეს შეიძლება იყოს VM, Docker host ან Kubernetes namespace. შეზღუდვაა მხოლოდ ის, რომ დღეს ერთი პროცესის რეჟიმით მუშაობს (G6), სანამ გადახედვის შენახვა ბაზაში არ გადავა.

💬 **მარტივად რომ გავიგოთ:** ცალკე სახლი არ გვჭირდება, ერთი ოთახიც საკმარისია. ოღონდ საკუთარი კარით, რომ მეზობლებს ხელი არ შევუშალოთ.

📌 **ჩვენი პროექტიდან:** ✅ ყველა პარამეტრი გარემოს ცვლადებშია, კოდში კონკრეტული სერვერი არ წერია.

❓ **გაურკვეველია:** ❓ IT-ის ვირტუალიზაციის ან კონტეინერის სტანდარტი.

➡️ **რა ვკითხოთ IT-ს:** „Python-ის ვებ-აპლიკაციები უკვე გაქვთ? რა გარემოში ეშვება?“

Q10. 🟡 რამდენი CPU და RAM?

🗣️ **მოკლე პასუხი:** „ჩვენი საწყისი შეფასებაა 4 vCPU და 8 GB RAM ერთ VM-ზე, PostgreSQL-ის ჩათვლით. ეს შეფასებაა და არა გაზომვა. დატვირთვის ტესტი ჯერ არ ჩაგვიტარებია. პილოტზე გავზომავთ და, თუ ნაკლები გვეყოფა, შევამცირებთ.“

🔧 **ტექნიკური ახსნა:**

- ≈ 300 მომხმარებელი, ერთდროულად ≈ 40 (შეფასება `IT_REQUEST.md`-დან);
- 2–4 uvicorn worker, თითოს რამდენიმე ასეული MB RAM (❓ გასაზომი);
- PostgreSQL-ს ქეშისთვის (`shared_buffers`) 1–2 GB;
- ყველაზე მძიმე ოპერაციებია სტატისტიკის აგრეგაცია (SQLite-ზე 2–3 წმ) და განაწილების გადახედვა (ათასობით ჩანაწერი მეხსიერებაში).

💬 **მარტივად რომ გავიგოთ:** ეს საშუალო ოფისის კომპიუტერის სიმძლავრეა. ასეთი აპლიკაციისთვის საკმარისი უნდა იყოს, მაგრამ „უნდა იყოს“ — ჯერ არ ვიცით.

📌 **ჩვენი პროექტიდან:** ✅ ბაზა 94 MB; ✅ ტესტები 6 წამში გადის. ❓ load test არ ყოფილა.

❓ **გაურკვეველია:** ❓ რეალური მოთხოვნების სიხშირე პიკის საათზე.

➡️ **რა ვკითხოთ IT-ს:** „VM-ის რესურსის შემდგომი გაზრდა მარტივია? მაშინ მცირით დავიწყოთ.“

Q11. 🟢 რამდენი დისკი დაგჭირდებათ?

🗣️ **მოკლე პასუხი:** „დღევანდელი ბაზა 44 ათას შემთხვევაზე 94 MB-ია. ზრდის ჩვენი შეფასებით, აპლიკაციის ბაზა წელიწადში 0.5–2 GB-ით გაიზრდება, ფოტოები 1–3 GB-ით, REGFIELDALL-ის ასლი კი 1–2 GB-ია. 100 GB რამდენიმე წლის მარაგია. ბექაპი ცალკე საცავში უნდა იყოს.“

🔧 **ტექნიკური ახსნა:**

- ✅ SQLite: 94 MB + WAL 4 MB (44 159 შემთხვევა, 128 690 მიკუთვნება, 45 765 კოდი, 44 159 მოვლენა);
- ✅ Distributor-ის ასლი: 11 ველი × 2.33 მლნ = ≈ 1 GB;
- 💡 ზრდა: მოვლენების რაოდენობა თითო შემთხვევაზე 5–10-მდე გაიზრდება. PostgreSQL-ის ზომა SQLite-ზე მეტია (tuple overhead, ინდექსები).

💬 **მარტივად რომ გავიგოთ:** ტექსტი ძალიან ცოტა ადგილს იკავებს. ფოტოები მეტს, მაგრამ მათაც ცოტა ტვირთავენ, რადგან ატვირთვა მხოლოდ კონტროლს შეუძლია.

📌 **ჩვენი პროექტიდან:** ✅ ფოტო ≤ 10 MB, მხოლოდ QC როლები.

❓ **გაურკვეველია:** ❓ ფოტოების რეალური რაოდენობა (დღეს 0 ატვირთვაა ბაზაში); ბექაპის შენახვის ვადა.

➡️ **რა ვკითხოთ IT-ს:** „ბექაპის საცავის მოცულობა ცალკე ითვლება თქვენთან?“

Q12. 🟢 Linux თუ Windows? Docker?

🗣️ **მოკლე პასუხი:** „ორივეზე იმუშავებს. Linux და Docker მხარდაჭერისთვის უფრო მარტივია, მაგრამ თქვენს სტანდარტს მივყვებით. დღეს Windows-ზე ვავითარებთ და ვამოწმებთ.“

🔧 **ტექნიკური ახსნა:**

- კოდი OS-დამოუკიდებელია (`pathlib` , გარემოს ცვლადები);
- Windows-სპეციფიკურია მხოლოდ `.bat` გამშვებები და `truststore` -ის მიერ Windows-ის სერტიფიკატების საცავის გამოყენება. Linux-ზე სისტემის საცავს იყენებს;
- Linux-ზე: systemd + nginx; Windows-ზე: სერვისი + IIS (ARR) ან nginx;

- gunicorn მხოლოდ Linux-ზე მუშაობს, `uvicorn --workers` კი ორივეზე;
- 💡 Dockerfile მოსამზადებელია.

💬 **მარტივად რომ გავიგოთ:** პროგრამა ორივე „ენაზე ლაპარაკობს“. Docker კი მზა „ყუთია“, სადაც საიტი ყველა ხელსაწყოსთან ერთად დევს.

📌 **ჩვენი პროექტიდან:** ✅ Python 3.14.5, Windows 11; `.venv` uv-ით.

❓ **გაურკვეველია:** ❓ IT-ის სტანდარტი; აქვთ თუ არა კონტეინერების რეესტრი.

➡️ **რა ვკითხოთ IT-ს:** „Docker image მოგაწოდოთ, თუ სერვერზე გარემოს თქვენ აწყობთ?“

Q13. 🔴 100 ადამიანი რომ ერთდროულად შევიდეს — გაუძლებს?

🗣️ **მოკლე პასუხი:** „ამ ტიპის აპლიკაციისთვის 100 ერთდროული მომხმარებელი ჩვეულებრივი დატვირთვაა. ადამიანები კითხულობენ და წერენ, ამიტომ წამში ათეულობით მოთხოვნაზე მეტი არ მოვა. მაგრამ ციფრით ვერ დაგიდასტურებთ, სანამ დატვირთვის ტესტს არ ჩავატარებთ. ტესტს პილოტამდე გავაკეთებთ, განსაკუთრებით სტატისტიკის გვერდზე, და შედეგს გაგიზიარებთ.“

🔧 **ტექნიკური ახსნა:**

- ერთდროული მომხმარებელი ≠ ერთდროული მოთხოვნა;
- ჩვეულებრივი გვერდი ასეულობით მილიწამია, სტატისტიკა 2–3 წმ (SQLite);
- PostgreSQL MVCC-ით მკითხველები მწერლებს არ ბლოკავენ;
- worker-ები × thread pool მოთხოვნებს პარალელურად ამუშავებს;
- რისკები:
  - სტატისტიკის მძიმე აგრეგაცია ერთდროულად → ბაზის CPU;
  - კავშირების აუზის ამოწურვა;
  - G6 — მრავალ worker-ზე გადასვლამდე გასასწორებელია.
- ტესტის გეგმა: k6 ან locust, სცენარები (ოპერატორის სია, ბარათი, გაგზავნა, სტატისტიკა), 50 / 100 / 200 ვირტუალური მომხმარებელი, p95 პასუხის დრო.

💬 **მარტივად რომ გავიგოთ:** 100 ადამიანი მაღაზიაში ერთ წამში ყველა სალაროსთან არ დგება. მაგრამ თუ ყველა ერთად მოითხოვს „მთელი თვის ანგარიშს“, ეს მძიმეა და გაზომვა სჭირდება.

📌 **ჩვენი პროექტიდან:** ✅ ერთდროული ცვლილების დაცვა: `version`, `uq_item_protocol_cad`. ⚠️ G5, G6, G14.

❓ **გაურკვეველია:** ❓ პიკის საათის რეალური პროფილი (მაგ. დილის 10-ზე ყველა ოპერატორი იწყებს).

➡️ **რა ვკითხოთ IT-ს:** „დატვირთვის ტესტის ჩატარება სატესტო გარემოზე შეიძლება? ამისთვის რა წესი გაქვთ?“

Q14. 🟡 სისტემა ინტერნეტის გარეშე იმუშავებს?

🗣 **მოკლე პასუხი:** „კი, შიდა ქსელში იმუშავებს: ყველა ბიბლიოთეკა, შრიფტი და რუკა სერვერზევეა, გარე CDN არ გამოიყენება. ინტერნეტი სერვერს მხოლოდ ორ რამეზე სჭირდება: Google-ით შესვლაზე (Google-ის სერვერებთან კავშირი) და ინსტალაციისას ბიბლიოთეკების ჩამოტვირთვაზე. ამ უკანასკნელის მოგვარება შიდა სარკით ან მზა image-ით შეიძლება.“

🔧 **ტექნიკური ახსნა:**

- ✅ `app/static/vendor/` : HTMX, ECharts, Leaflet, Noto Sans Georgian; ✅ `app/static/geo/zones.geojson` ;
- გამავალი კავშირი: `accounts.google.com` (ბრაუზერიდან), `oauth2.googleapis.com` (სერვერიდან, 443);
- SMTP: შიდა relay, ინტერნეტი არ სჭირდება;
- ველის მომხმარებლებს (ტელეფონი) სერვერზე წვდომა სჭირდებათ: საჯარო HTTPS ან VPN.

💬 **მარტივად რომ გავიგოთ:** საიტი თავისი „ავეჯით“ მოდის, გარედან არაფერს იწერს. შესვლისთვის მხოლოდ Google-ს „ეკითხება“, ვინ ხარ.

📌 **ჩვენი პროექტიდან:** ✅ `ARCHITECTURE.md` §3: „გარე CDN არ გამოიყენება“.

❓ **გაურკვეველია:** ❓ თუ სერვერს Google-თან გამავალი კავშირი სრულად აეკრძალება, Google OAuth ვერ იმუშავებს. მაშინ ალტერნატიული შესვლაა საჭირო (Q45).

➡️ **რა ვკითხოთ IT-ს:** „სერვერიდან `oauth2.googleapis.com:443` -ზე გამავალ კავშირს დაუშვებთ? proxy-ით, თუ პირდაპირ?“

Q15. 🟡 ველიდან, ტელეფონით, გარედან როგორ შევლენ?

🗣 **მოკლე პასუხი:** „ორი გზა არსებობს: საჯარო HTTPS მისამართი, ან VPN ტელეფონზე. ველისთვის საჯარო HTTPS უფრო პრაქტიკულია, Google-ის შესვლით და მხოლოდ დარეგისტრირებული ელფოსტებით. არჩევანი თქვენი უსაფრთხოების პოლიტიკისაა. აპლიკაცია ორივეზე ერთნაირად მუშაობს.“

🔧 **ტექნიკური ახსნა:**

- საჯარო გამოქვეყნება: reverse proxy / WAF DMZ-ში → შიდა აპლიკაციის სერვერი;
- დამატებითი დაცვა: rate limiting, WAF-ის წესები, HSTS, შესაძლოა გეო-ფილტრი;
- VPN: ტელეფონზე კლიენტი, მართვის ტვირთი ≈ 250 ამზომველზე;
- ✅ ინტერფეისი მობილურზე მუშაობს: ველის სია ბარათებად, LAN დემოთი გამოცდილია.

💬 **მარტივად რომ გავიგოთ:** ან ქუჩაში დაცული კარი ვაკეთებთ (საჯარო HTTPS), ან ყველას ტელეფონზე „გვირაბის“ გასაღებს ვაძლევთ (VPN). 250 ტელეფონზე გვირაბი რთულია.

📌 **ჩვენი პროექტიდან:** ✅ `run_lan.bat` — ტელეფონიდან ჩვენება იმავე Wi-Fi-ში, PIN-ით.

❓ **გაურკვეველია:** ❓ სააგენტოს პოლიტიკა შიდა აპლიკაციების გარეთ გამოქვეყნებაზე.

➡️ **რა ვკითხოთ IT-ს:** „მსგავსი აპლიკაციები, რომლებიც ველიდან გამოიყენება, როგორ გაქვთ გამოქვეყნებული?“

Q16. 🟢 დომენი და HTTPS რატომ არის სავალდებულო?

🗣️ **მოკლე პასუხი:** „Google-ით შესვლას HTTPS მისამართი სჭირდება: Google სხვა მისამართზე მომხმარებელს არ დააბრუნებს. გარდა ამისა, სესიის cookie და QC მონაცემი ქსელში დაშიფრული უნდა მოძრაობდეს, განსაკუთრებით ტელეფონიდან.“

🔧 **ტექნიკური ახსნა:**

- Google OAuth-ის Authorized redirect URI HTTPS-ს მოითხოვს (localhost-ის გარდა);
- ✅ კოდი: `redirect_uri = QCP_BASE_URL + "/auth/google/callback"`;
- ✅ cookie-ს `Secure` ნიშანი;
- ⚠️ proxy-ს უკან uvicorn-ს `--proxy-headers` სჭირდება (G4);
- TLS 1.2+.

💬 **მარტივად რომ გავიგოთ:** Google „ცნობას“ (შესვლის დადასტურებას) მხოლოდ დალუქულ კონვერტში და მხოლოდ ზუსტ მისამართზე გამოგზავნის.

📌 **ჩვენი პროექტიდან:** ✅ `app/routes/google_auth.py`.

❓ **გაურკვეველია:** ❓ სერტიფიკატის წყარო (შიდა CA თუ საჯარო).

➡️ **რა ვკითხოთ IT-ს:** „ქვედომენის და სერტიფიკატის გაცემას რა პროცედურა აქვს?“

Q17. 🟡 რა პორტები უნდა გავხსნათ?

🗣️ **მოკლე პასუხი:** „შემომავალი: მხოლოდ 443 proxy-ზე (და 80, HTTPS-ზე გადამისამართებისთვის). შიგნით: proxy → აპლიკაცია და აპლიკაცია → PostgreSQL. გამავალი: Google-ის OAuth 443-ზე და SMTP relay. წყაროებზე: არჩეული გზის მიხედვით.“

🔧 **ტექნიკური ახსნა:**

საიდან → სად	პორტი	რისთვის
მომხმარებლები → proxy	443 (80 → 301)	საიტი
proxy → აპლიკაცია	8200 (ან სოკეტი; localhost)	—
აპლიკაცია → PostgreSQL	5432 (TLS, თუ ცალკე VM-ზეა)	ბაზა
აპლიკაცია → `oauth2.googleapis.com`	443	token-ის გაცვლა

საიდან → სად	პორტი	რისთვის
აპლიკაცია → SMTP relay	587 / 25 / თქვენი	მეილი
worker → REGFIELDALL view / ფაილი	5432 / 1433 / 445 (SMB) / 22 (SFTP)	❓ არჩევანზეა დამოკიდებული
worker → WFS (Pre-QC, მომავალში)	443	❓
ადმინისტრირება → სერვერი	22 (SSH) / 3389 (RDP), მხოლოდ შიდა ქსელიდან	deployment

💬 **მარტივად რომ გავიგოთ:** ქუჩიდან მხოლოდ ერთი კარია (443). დანარჩენი შიდა კარებია, და თითოს მხოლოდ კონკრეტული „მცხოვრები“ იყენებს.

📌 **ჩვენი პროექტიდან:** ✅ `QCP_PORT` = 8200.

❓ **გაურკვეველია:** ❓ წყაროს წვდომის გზა.

➡️ **რა ვკითხოთ IT-ს:** „firewall-ის მოთხოვნა რა ფორმით გინდათ? ცხრილი ზემოთ გამოგადგებათ?“

Q18. 🟡 განახლებებს როგორ დააყენებთ მომხმარებლების შეფერხების გარეშე?

🗣️ **მოკლე პასუხი:** „ჩვეულებრივი განახლება წამებს გრძელდება: სესიები ბაზაშია, ამიტომ მომხმარებელი სისტემიდან არ „ვარდება“. რამდენიმე worker-ით მორიგეობითი რესტარტი შეფერხებას საერთოდ მოხსნის. ბაზის სტრუქტურის ცვლილება ვერსიირებული მიგრაციებით ხდება: ჯერ სატესტო გარემოზე, ბექაპით, და შეთანხმებულ დროს.“

🔧 **ტექნიკური ახსნა:**

- deployment: tag → ტესტები → ბექაპი → `alembic upgrade` (💡 G2) → rolling restart → smoke test (`scripts/smoke_pages.py`) → rollback გეგმა;
- თავსებადი მიგრაციები (expand → migrate → contract), რომ ძველმა კოდმა ახალ სქემაზეც იმუშაოს;
- ⚠️ G6: მეხსიერებაში შენახული გადახედვა რესტარტზე იკარგება. QC მენეჯერს გადახედვის ხელახლა გაშვება მოუწევს, სანამ ეს ბაზაში არ გადავა.

💬 **მარტივად რომ გავიგოთ:** მაღაზიას ღამით ან შესვენებაზე ვარემონტებთ, ერთ-ერთ სალაროს მორიგეობით ვკეტავთ, და მყიდველი ვერც ამჩნევს.

📌 **ჩვენი პროექტიდან:** ✅ სესიები `sessions` ცხრილში. ✅ smoke-სკრიპტები ყველა გვერდს ყველა როლით ამოწმებს.

❓ **გაურკვეველია:** ❓ IT-ის change management პროცესი, ცვლილების ფანჯრები.

➡️ **რა ვკითხოთ IT-ს:** „განახლების დამტკიცების პროცედურა გაქვთ? სატესტო გარემოს მოგვცემთ?“

Q19. 🟢 მონიტორინგს როგორ აპირებთ?

🗣️ **მოკლე პასუხი:** „თქვენს მონიტორინგის სისტემაში ჩართვა გვირჩევნია. ჩვენ გვაქვს health endpoint (`/api/health`), აუდიტი და იმპორტის ჟურნალი ბაზაში. დავამატებთ აპლიკაციის ლოგს ფაილში, და განგაშს იმპორტის ჩავარდნაზე.“

🔧 **ტექნიკური ახსნა:**

- ✅ `/api/health` → `{"status": "ok" | "empty"}` . ⚠️ ბაზის მიუწვდომლობისას 500-ს აბრუნებს. 💡 გასათვალისწინებელია `200 + degraded` , როგორც Pre-QC hosted-შია;
- ⚠️ G16: `log_level="warning"` ;
- 💡 მეტრიკები: პასუხის დრო, 5xx-ის სიხშირე, DB კავშირები, იმპორტის ასაკი, დისკი;
- 💡 ლოგში არ უნდა ჩაიწეროს OAuth `code` , cookie და პირადი ნომერი.

💬 **მარტივად რომ გავიგოთ:** სერვერს „პულსს“ ყოველ წუთს ვუმოწმებთ. თუ გაჩერდა, ვიღაცას ეუწყება.

📌 **ჩვენი პროექტიდან:** ✅ `audit_log` , `import_runs` ცხრილები.

❓ **გაურკვეველია:** ❓ IT-ის მონიტორინგის ინსტრუმენტი (Zabbix, Prometheus, ...).

➡️ **რა ვკითხოთ IT-ს:** „ლოგები და მეტრიკები რა ფორმატით გინდათ?“

Q20. 🟡 სერვერი რომ ჩავარდეს, რა მოხდება? მაღალი ხელმისაწვდომობა (HA) გჭირდებათ?

🗣️ **მოკლე პასუხი:** „დადასტურებული ცვლილება არ იკარგება: ბაზა ტრანზაქციულია. იკარგება მხოლოდ ის მოთხოვნა, რომელიც იმ წამს მუშავდებოდა. HA-ს ორი სერვერით პირველ ეტაპზე არ ვითხოვთ. QC რამდენიმე საათიან შეფერხებას გაუძლებს, ოღონდ საჭიროა ბექაპი, აღდგენის გეგმა და სერვისის ავტომატური რესტარტი.“

🔧 **ტექნიკური ახსნა:**

- PostgreSQL WAL-ით აღდგება;
- systemd/Docker restart პოლიტიკა;
- 💡 RPO ≤ 15 წთ (PITR), RTO ≤ 4 სთ სამუშაო დღეს (შეთავაზება, ❓ დასამტკიცებელი);
- HA (რეპლიკა + failover) შემდეგი ეტაპისთვის.

💬 **მარტივად რომ გავიგოთ:** შენახული არ იკარგება. თუ სერვერი გაფუჭდა, ბექაპიდან ავადგენთ. ორი სერვერი ჯერ ფუფუნებაა.

📌 **ჩვენი პროექტიდან:** ✅ ყოველი მოქმედება ერთ ტრანზაქციაშია. ✅ Distributor-იც ქსელის ჩავარდნისას ბოლო ჩატვირთულ მონაცემზე მუშაობს.

❓ **გაურკვეველია:** ❓ ხელმძღვანელობის მოთხოვნა დასაშვებ შეფერხებაზე.

➡️ **რა ვკითხოთ IT-ს:** „ჩვეულებრივ, რა RPO/RTO გაქვთ შიდა აპლიკაციებზე?“

გ. მონაცემთა ბაზა

Q21. 🔴 რატომ არ იყენებთ ჩვენს არსებულ ბაზას? ახალი ბაზა რატომ?

🗣 **მოკლე პასუხი:** „თქვენს სისტემებს მხოლოდ ვკითხულობთ და მათში არაფერს ვწერთ, რომ მათ არ შევეხოთ და არ დავტვირთოთ. ჩვენს სამუშაო მონაცემს (ვერდიქტები, ველის პასუხები, ისტორია, კომენტარები) საკუთარი ადგილი სჭირდება. ეს შეიძლება იყოს ცალკე ბაზა თქვენს არსებულ DB სერვერზე. ახალი სერვერი სავალდებულო არ არის.“

🔧 **ტექნიკური ახსნა:**

- **ნაკადების გამიჯვნა:** წყარო (system of record ოქმებზე) და QC (system of record QC გადაწყვეტილებებზე) სხვადასხვაა;
- **ფიზიკურად შეუძლებელია წყაროს შეცვლა:** ჩვენი ანგარიში წყაროზე მხოლოდ `SELECT`-ს იღებს;
- **დამოუკიდებელი სასიცოცხლო ციკლი:** ჩვენი სქემის მიგრაცია თქვენს ბაზას არ ეხება;
- **დატვირთვა:** ჩვენი სტატისტიკის შეკითხვები თქვენს OLTP-ს არ ტვირთავს.

💬 **მარტივად რომ გავიგოთ:** თქვენი არქივიდან ასლს ვიღებთ, ჩვენს სამუშაოს კი ჩვენს რვეულში ვწერთ. ასე თქვენს არქივში შემთხვევით ვერაფერს გავაფუჭებთ.

📌 **ჩვენი პროექტიდან:** ✅ „მხოლოდ კითხვა“ პრინციპი (`IT_REQUEST.md` §2). ✅ Distributor-ის ბაზა `mode=ro`-ით იხსნება.

❓ **გაურკვეველია:** ❓ IT-ს აქვს თუ არა PostgreSQL ან SQL Server კლასტერი, სადაც ახალი ბაზა შეიძლება.

➡️ **რა ვკითხოთ IT-ს:** „ახალი ბაზის შექმნა თქვენს არსებულ DB სერვერზე შეიძლება? რა ძრავაა?“

Q22. 🔴 რატომ PostgreSQL და არა Microsoft SQL Server?

🗣 **მოკლე პასუხი:** „PostgreSQL უფასოა, ჩვენი Pre-QC-ის სერვერული ნაწილი უკვე მასზეა, და გამოცდილებაც გვაქვს. მაგრამ აპლიკაცია ORM-ზეა აგებული, და SQL Server-ზე გადატანა შესაძლებელია. თუ თქვენი სტანდარტი და DBA SQL Server-ისაა, ტესტებს მასზე გავუშვებთ და განსხვავებებს გავასწორებთ.“

🔧 **ტექნიკური ახსნა:**

- SQLAlchemy ორივე დიალექტს უჭერს მხარს (`psycopg` / `pyodbc`);
- SQL Server-ზე შესამოწმებელია:
  - JSON სვეტები (`NVARCHAR(MAX)`);
  - ქართული collation;
  - თარიღის ფუნქციები სტატისტიკაში;
  - ინდექსის სახელების სიგრძე;
- Express ვერსიას ბაზის ზომის ლიმიტი აქვს (10 GB);
- PostgreSQL-ის უპირატესობები: `JSONB` , `ON CONFLICT` upsert, `DISTINCT ON` , Pre-QC hosted-ის მიგრაციები და ტრიგერები.

💬 **მარტივად რომ გავიგოთ:** ორივე კარგი „სეიფია“. ერთი უფასოა და უკვე ვიცნობთ, მეორე შეიძლება თქვენ უკვე გქონდეთ და უვლიდეთ. ორივე გამოდგება.

📌 **ჩვენი პროექტიდან:** ✅ `app/db.py` — SQLite-ის სპეციფიკა მხოლოდ `if _is_sqlite` -შია. ✅ `hosted/README.md` — „SQLite განზრახ დაბლოკილია... სქემა იყენებს JSONB-ს, INET-ს, მასივებსა და ტრიგერებს“.

❓ **გაურკვეველია:** ❓ ⚠️ არცერთ სერვერულ ბაზაზე ტესტი ჯერ არ გაგვიშვია (G1).

➡️ **რა ვკითხოთ IT-ს:** „SQL Server თქვენი სავალდებულო სტანდარტია, თუ PostgreSQL-იც მისაღებია? DBA-ს მხარდაჭერა რომელზე გაქვთ?“

Q23. 🟡 SQLite-ზე ხომ მუშაობს — საწარმოოშიც რატომ არ დარჩება?

🗣️ **მოკლე პასუხი:** „SQLite ერთ ფაილში ინახავს ყველაფერს და ერთდროულად ერთ მწერალს უშვებს. ერთი პროცესისა და დემოსთვის იდეალურია, 300 მომხმარებლისთვის, რამდენიმე worker-ით და ბექაპით კი — არა. ქსელურ საქაღალდეზე SQLite-ის ფაილის დადება ასევე არასანდოა.“

🔧 **ტექნიკური ახსნა:**

- SQLite-ში ჩაწერა მთელი ბაზის დონეზე ბლოკდება (WAL-ში მკითხველები არ ბლოკდებიან, მაგრამ მწერალი ერთია);
- არ აქვს ქსელური წვდომა, მომხმარებლები და როლები;
- PITR შეუძლებელია;
- ✅ ჩვენ WAL, `synchronous=NORMAL` და `timeout=30` ვიყენებთ, რაც დემოსთვის საკმარისია.

💬 **მარტივად რომ გავიგოთ:** პირადი რვეული ერთი ადამიანისთვის კარგია. 300 ადამიანმა რიგში რომ წეროს ერთ რვეულში, ნელა იქნება, და რვეულის დაკარგვა ყველაფრის დაკარგვაა.

📌 **ჩვენი პროექტიდან:** ✅ `app/db.py` : `PRAGMA journal_mode=WAL` .

❓ **გაურკვეველია:** —

➡️ **რა ვკითხოთ IT-ს: —**

Q24. 🟢 ბექაპი და აღდგენა როგორ იქნება?

🗣️ **მოკლე პასუხი:** „ბაზის ბექაპი PITR-ით გვინდა: base backup და WAL-ის არქივი, რომ ნებისმიერ წუთზე აღდგენა შეიძლებოდეს, პლუს ფოტოების საქაღალდის ბექაპი. იდეალურია თქვენი სტანდარტული ბექაპის სისტემით, სხვა ლოკაციაზე. აღდგენის ტესტი კვარტალში ერთხელ მაინც.“

🔧 **ტექნიკური ახსნა:**

- PostgreSQL: `pg_basebackup` + `archive_command`, ან pgBackRest/Barman. მინიმუმ ღამის `pg_dump -Fc`;
- 3-2-1 წესი, დაშიფვრა;
- ფოტოები: `data/uploads/` — ფაილური ბექაპი იმავე განრიგით;
- კონფიგურაცია: გარემოს ფაილი, საიდუმლოებების გარეშე, ან საიდუმლოების საცავის ბექაპი;
- აღდგენის ტესტი: სატესტო ბაზაში აღდგენა → აპლიკაციის გაშვება → smoke test.

💬 **მარტივად რომ გავიგოთ:** ბაზის „ვიდეოჩანაწერი“ ყოველ წუთზე, რომ შეცდომამდე წუთზე დავბრუნდეთ. ფოტოების „ასლი“ ყოველ ღამე. და დროდადრო ვამოწმებთ, რომ ჩანაწერი მართლა იკითხება.

📌 **ჩვენი პროექტიდან:** ✅ დღეს ბექაპი არ კეთდება. ⚠️ `build_db.py` აღდგენა არ არის: ბაზას ძველი ცხრილიდან ნულიდან აწყობს.

❓ **გაურკვეველია:** ❓ IT-ის ბექაპის ინსტრუმენტი და შენახვის ვადა.

➡️ **რა ვკითხოთ IT-ს:** „PostgreSQL-ის ბექაპს თქვენი სისტემა უჭერს მხარს? PITR შესაძლებელია?“

Q25. 🟡 ბაზაზე რა უფლებები გჭირდებათ?

🗣️ **მოკლე პასუხი:** „ჩვენს ბაზაზე ორი მომხმარებელი: აპლიკაციის (ჩვეულებრივი წაკითხვა და ჩაწერა, სქემის შეცვლის გარეშე) და მიგრაციის (სქემის ცვლილებისთვის, მხოლოდ განახლებისას). superuser არ გვჭირდება. თქვენს ბაზებზე მხოლოდ `SELECT`, მხოლოდ ჩვენთვის შექმნილ view-ებზე.“

🔧 **ტექნიკური ახსნა:**

- `qc_app`: `SELECT, INSERT, UPDATE, DELETE` ჩვენს სქემაზე. 💡 `REVOKE UPDATE, DELETE ON qc_events` — ისტორია მხოლოდ ემატება;
- `qc_migrator`: სქემის მფლობელი;
- `qc_reader` (წყაროზე): `GRANT SELECT ON <view>`;
- კავშირი მხოლოდ აპლიკაციის სერვერის IP-დან (`pg_hba.conf` / firewall).

💬 **მარტივად რომ გავიგოთ:** საიტს „კარადის“ გამოყენება შეუძლია, მაგრამ „კარადის დანგრევა“ — არა. თქვენი სისტემიდან კი მხოლოდ ფანჯარაში ვიყურებით.

📌 **ჩვენი პროექტიდან:** სახელმძღვანელო B.14 — SQL-ის მაგალითი.

❓ **გაურკვეველია:** ⚠️ ისტორიის `REVOKE`-მდე შესამოწმებელია, რომ საწარმოოს არცერთი კოდის გზა `qc_events`-ს არ ცვლის. ძველი მონაცემის ხელახალი იმპორტი მას შლის.

➡️ **რა ვკითხოთ IT-ს:** „ბაზის მომხმარებლების შექმნა თქვენი DBA-ს საქმეა? პაროლებს სერვერზე პირდაპირ ჩაწერთ?“

Q26. 🟢 როგორ იცავთ მონაცემის მთლიანობას? დუბლიკატები?

🗣 **მოკლე პასუხი:** „ბაზის დონეზე: ოქმის ნომერი და საკადასტრო კოდი ერთად უნიკალურია, ამიტომ ერთი ოქმის ორჯერ განაწილება ფიზიკურად შეუძლებელია. მდგომარეობები და ვერდიქტები შემოწმების შეზღუდვებით არის დაცული, კავშირები — foreign key-ებით. ყოველი მოქმედება ერთ ტრანზაქციაშია: ან ყველაფერი ინახება, ან არაფერი.“

🔧 **ტექნიკური ახსნა:**

- ✅ `uq_item_protocol_cad`, `uq_backlog_item`;
- ✅ `ck_item_state`, `ck_item_verdict`, `ck_item_kind`, `ck_user_role`, `ck_person_kind`;
- ✅ FK + `PRAGMA foreign_keys=ON`;
- ✅ `users.email UNIQUE`;
- ✅ ნორმალიზაცია შენახვამდე (ჰარები, ოქმის ნომრის `float → int → str`).

💬 **მარტივად რომ გავიგოთ:** ბაზას თავისი „დაცვის თანამშრომელი“ ჰყავს: აპლიკაციამ რომც შეცდეს, ბაზა არასწორს არ ჩაწერს.

📌 **ჩვენი პროექტიდან:** ✅ ძველ ცხრილში 379 დუბლიკატი იყო. ახალში ასეთი ჩანაწერი ვერ გაჩნდება.

❓ **გაურკვეველია:** —

➡️ **რა ვკითხოთ IT-ს:** —

Q27. 🟡 ორი ადამიანი ერთ ჩანაწერს რომ ცვლიდეს?

🗣 **მოკლე პასუხი:** „თითო ოქმს ვერსიის ნომერი აქვს. თუ ორმა ერთდროულად გააგზავნა, პირველი მიიღება, მეორე კი ხედავს: „უკვე უპასუხა ამა და ამ ადამიანმა, ამ დროს“. ამას ემატება ყვითელი ნიშანი „ახლა ხსნის: X“, რომელიც წინასწარ აფრთხილებს. PostgreSQL-ზე გადასვლისას ამ შემოწმებას ბაზის დონეზე გადავიტანთ.“

🔧 **ტექნიკური ახსნა:**

- ✅ ოპტიმისტური ბლოკირება `qc_items.version`-ით, `workflow.apply()`-ში;

- ✅ presence: ping 20 წმ, აქტიური 60 წმ;
- ⚠️ G5: შედარება აპლიკაციაშია, `UPDATE … WHERE version = ?` არ ხდება, და ფორმის გარეშე `version` შემოწმება გამოტოვდება. 💡 SQLAlchemy `version_id_col` ან `SELECT … FOR UPDATE`.

💬 **მარტივად რომ გავიგოთ:** ოქმს „მრიცხველი“ აქვს. თუ ბარათის გახსნიდან გაგზავნამდე მრიცხველი შეიცვალა, ესე იგი სხვამ დაგასწრო, და შენი ცვლილება არ შევა.

📌 **ჩვენი პროექტიდან:** ✅ `routes/items.action`: „ამ ოქმზე პასუხი უკვე გაგზავნა … — ერთ კოდზე ერთი პასუხი იგზავნება“. ✅ ტესტი `test_workflow.py`.

❓ **გაურკვეველია:** —

➡️ **რა ვკითხოთ IT-ს:** —

Q28. 🟡 სქემის ცვლილებას საწარმოოში როგორ გააკეთებთ?

🗣️ **მოკლე პასუხი:** „ვერსიირებული მიგრაციებით (Alembic). ყოველი ცვლილება ფაილადაა ჩაწერილი, აქვს დაბრუნების ნაბიჯი, ჯერ სატესტო გარემოზე მოწმდება და ბექაპის შემდეგ ეშვება. დღეს პროტოტიპი ცხრილებს გაშვებისას ქმნის. ამის შეცვლა ჩვენი პირველი სამუშაოა საწარმოომდე.“

🔧 **ტექნიკური ახსნა:**

- ⚠️ G2: `Base.metadata.create_all(engine)` — ახალ ცხრილს ქმნის, არსებულს არ ცვლის;
- 💡 Alembic: საწყისი მიგრაცია დღევანდელი სქემით, შემდეგ `revision --autogenerate` + ხელით გადახედვა;
- ✅ Pre-QC hosted-ში Alembic უკვე გამოიყენება.

💬 **მარტივად რომ გავიგოთ:** ბაზის „რემონტი“ ნაბიჯ-ნაბიჯ, წერილობითი გეგმით, და არა „ავდექით და შევცვალეთ“.

📌 **ჩვენი პროექტიდან:** ✅ `app/main.py`, მე-32 ხაზი.

❓ **გაურკვეველია:** —

➡️ **რა ვკითხოთ IT-ს:** „მიგრაციის გაშვება ჩვენ შეგვიძლია, თუ თქვენი DBA უშვებს?“

Q29. 🟡 მონაცემს რამდენ ხანს შეინახავთ?

🗣️ **მოკლე პასუხი:** „ტექნიკურ ჩანაწერებს მოკლე ვადას ვთავაზობთ: სესიები 30 დღე, მეილის ასლი 1 წელი. QC ჩანაწერებისა და აუდიტის ვადა სააგენტოს საარქივო და პერსონალური მონაცემის წესით უნდა განისაზღვროს. ამაზე თქვენი და იურიდიული სამსახურის პოზიცია გვჭირდება.“

🔧 **ტექნიკური ახსნა:** ✅ დღეს მხოლოდ `presence` იშლება (10 წთ), და სპორადულის ატვირთული ფაილი (მაშინვე). დანარჩენი სამუდამოდ ინახება. 💡 ღამის გასუფთავების

ამოცანა.

💬 **მარტივად რომ გავიგოთ:** ყველაფერი სამუდამოდ არ უნდა ინახებოდეს. რა რამდენ ხანს, ეს წესი სააგენტომ უნდა თქვას.

📌 **ჩვენი პროექტიდან:** სახელმძღვანელო B.15.

❓ **გაურკვეველია:** ❓ სააგენტოს საარქივო წესი.

➡️ **რა ვკითხოთ IT-ს:** „მსგავს ჩანაწერებზე შენახვის ვადის სტანდარტი გაქვთ?“

Q30. 🟢 ბაზას ვინ უვლის? რამდენად გაიზრდება?

🗣️ **მოკლე პასუხი:** „ბაზის ექსპლუატაციას (ბექაპი, პაჩები, მონიტორინგი) თქვენთან ვთავაზობთ, სტრუქტურასა და შინაარსს — ჩვენთან. ზრდა მცირეა: წელიწადში 0.5–2 GB-ია ჩვენი შეფასება.“

🔧 **ტექნიკური ახსნა:** იხ. Q11. PostgreSQL-ის რუტინა: autovacuum (ჩართულია ნაგულისხმევად), `pg_stat_statements` ნელი შეკითხვების სანახავად, მინორ ვერსიების განახლება.

💬 **მარტივად რომ გავიგოთ:** „სეიფის“ ტექნიკური მომსახურება თქვენ, „საქაღალდეების“ წესრიგი ჩვენ.

📌 **ჩვენი პროექტიდან:** ✅ 94 MB / 44 ათასი შემთხვევა.

❓ **გაურკვეველია:** —

➡️ **რა ვკითხოთ IT-ს:** „DBA-ს სერვისი შიდა აპლიკაციებზე გაქვთ?“

Q31. 🔴 თქვენს დოკუმენტში წერია „PostgreSQL-ზე გადასასვლელად მხოლოდ URL იცვლება“. ეს შემოწმებულია?

🗣️ **მოკლე პასუხი:** „გულწრფელად: არა. ეს ORM-ის შესაძლებლობაა და კოდიც ასე დავწერეთ, მაგრამ PostgreSQL-ზე ჯერ არ გაგვიშვია: დრაივერიც არ არის დაყენებული. გადასვლა და ტესტების გაშვება PostgreSQL-ზე ჩვენი სამუშაოა პილოტამდე. თუ რამე დიალექტის სხვაობა გამოჩნდება, იქ გავასწორებთ.“

🔧 **ტექნიკური ახსნა:**

- ⚠️ C3 / G1: `requirements.txt`-ში `psycopg` არ არის;
- ✅ ტიპები პორტატულია (`JSON` , `String` , `DateTime`); SQLite-ის `PRAGMA`-ები გამოყოფილია;
- შესამოწმებელი რისკები:
  - `CheckConstraint`-ების ტექსტი;
  - `func.lower` / `like` ქართულ ტექსტზე;
  - `JSON` → `JSONB` ;

- naive `datetime` vs `timestamptz` (G10);
- `distribution.source` SQLite-ის `sqlite3` მოდულით მუშაობს — ეს წყაროა და შეიცვლება.

💬 **მარტივად რომ გავიგოთ:** „ძრავის შეცვლა შესაძლებელია“, მაგრამ ჯერ არ გამოგვიცდია. ჯერ გამოვცდით და მერე გეტყვით, რომ მზადაა.

📌 **ჩვენი პროექტიდან:** ✅ `app/db.py`, `requirements.txt`.

❓ **გაურკვეველია:** ❓ რამდენ სამუშაოს მოითხოვს. სავარაუდოდ მცირეს, მაგრამ გაზომვამდე ამას ვერ დავპირდები.

➡️ **რა ვკითხოთ IT-ს:** „სატესტო PostgreSQL-ს ადრე მოგვცემთ, სანამ საწარმოო სერვერი მზადდება? პარალელურად ვიმუშავებდით.“

დ. ინტეგრაცია და მონაცემი

Q32. 🟢 ჩვენი სისტემებიდან რა მონაცემი გჭირდებათ?

🗣️ **მოკლე პასუხი:** „REGFIELDALL-იდან შვიდი ველი: ოქმის ნომერი, საკადასტრო კოდი, შექმნის თარიღი, საველე სტატუსი, ოქმის შემდგენი, აზომვის შემდგენი და ბოლო ჩანაწერის ნიშანი. სპორადულის სიიდან ათი სვეტი, განმცხადებლების მონაცემის გარეშე. მოგვიანებით, Pre-QC-ისთვის, ოქმის დეტალები და რუკის ფენები. პირადი ნომრები და ტელეფონები არ გვჭირდება.“

🔧 **ტექნიკური ახსნა:**

- REGFIELDALL: `FLD_PAPER_ID, CADCODE, MOX_CREATE_DATE, FLD_STATUS_NAME, OQMI_CREATOR, MOXAZ_CREATOR, IS_LAST`;
- სპორადული: `FLD_PAPER_ID, CADCODE, CREATE_DATE, SURVEY_DAY, SURVEY_TIME, CALL_STATUS, ST_NAME, TR_NAME, USER_REAL_NAME, REGNUMBER`;
- ოქმის დეტალები: შემსრულებლები სტატუსით „საველე სამუშაოების შესრულებაზე უფლებამოსილი პირი“, დანართების სია;
- WFS: `SLR:GFLD_PARCELS` და სხვ.

💬 **მარტივად რომ გავიგოთ:** მხოლოდ ის ინფორმაცია გვჭირდება, რაც ოქმის „ვინაობას“ და „ვინ გააკეთა“-ს ამბობს. სხვისი პირადი ინფორმაცია არა.

📌 **ჩვენი პროექტიდან:** ✅ `services/sporadic.py`: `NEEDED`; ✅ `IT_REQUEST.md` §3.

❓ **გაურკვეველია:** ❓ ოქმის დეტალების წყარო და ფორმატი.

➡️ **რა ვკითხოთ IT-ს:** „ეს ველები ერთ ცხრილში ან view-შია, თუ რამდენიმე სისტემაშია გაბნეული?“

Q33. 🟡 view, API თუ ფაილი — რომელი გინდათ?

🗣 **მოკლე პასუხი:** „ჩვენი რიგი ასეთია: ჯერ read-only view, მერე REST API, მერე ღამის ფაილი დაცულ საქაღალდეში ან SFTP-ზე. view ყველაზე ნაკლებ სამუშაოს მოითხოვს თქვენგან და ყოველთვის ახალ მონაცემს იძლევა. თუ ბაზაზე წვდომა თქვენი პოლიტიკით არ შეიძლება, დღევანდელი ღამის GDB ექსპორტიც გამოდგება.“

🔧 **ტექნიკური ახსნა:**

გზა	IT-ის სამუშაო	სიახლე	დატვირთვა წყაროზე	ჩვენი სამუშაო
view	`CREATE VIEW` + read-only როლი + firewall	მოთხოვნისას	ღამის შეკითხვა	upsert-ის კონექტორი
API	თუ უკვე არსებობს — მცირე; თუ არა — დიდი	მოთხოვნისას	კონტროლირებადი (rate limit)	HTTP კონექტორი
ფაილი	ექსპორტი უკვე არსებობს (GDB)	ღამის	არ არის	GDAL ან CSV-ის წამკითხველი

💬 **მარტივად რომ გავიგოთ:**

- view — პატარა ფანჯარა თქვენს არქივში;
- API — ფანჯარა, სადაც თანამშრომელი გაწვდის;
- ფაილი — ამანათი ღამით.

სამივე მუშაობს, განსხვავება სიმარტივესა და სიახლეშია.

📌 **ჩვენი პროექტიდან:** ✅ დღეს ფაილის გზა მუშაობს (`Z:\REGFIELDALL`).

❓ **გაურკვეველია:** ❓ რომელი არის დასაშვები IT-ის პოლიტიკით.

➡️ **რა ვკითხოთ IT-ს:** „ამ სამიდან რომელი შეგიძლიათ ყველაზე სწრაფად? პილოტისთვის დღევანდელი GDB საქაღალდე სერვერზე მივაბათ?“

Q34. 🟡 მონაცემი როგორ სინქრონიზდება?

🗣 **მოკლე პასუხი:** „ღამით, ერთხელ, ფონური ამოცანით. ის წყაროს კითხულობს, ჩვენს ასლს ანახლებს (ახალს ამატებს, შეცვლილს ცვლის), და ყოველი გაშვება ჟურნალში იწერება. საჭიროების შემთხვევაში ადმინს დღისით ხელითაც შეუძლია გაშვება. განაწილების გვერდზე ყოველთვის ჩანს, რა თარიღის მონაცემზეა.“

🔧 **ტექნიკური ახსნა:**

- upsert `ON CONFLICT (fld_paper_id, cadcode)`;
- იდემპოტენტური;
- `import_runs`: დაწყება, დასრულება, რაოდენობა, შეცდომა;

- advisory lock, რომ ორი ასლი არ გაეშვას;
- „მოძველებულის“ ნიშანი და არა წაშლა;
- ⚠️ დღეს `build_db.py` ნულიდან აწყობს — საწარმოოში დაუშვებელია (G11).

💬 **მარტივად რომ გავიგოთ:** ყოველ ღამე „სია“ ახლდება: ახალი ხაზები ემატება, შეცვლილი სწორდება, არაფერი იშლება. დილით ყველა ახალ სიაზე მუშაობს.

📌 **ჩვენი პროექტიდან:** ✅ `source.source_info()` გვერდზე GDB-ის სახელს და იმპორტის თარიღს აჩვენებს.

❓ **გაურკვეველია:** ❓ რომელ საათზე იქმნება ექსპორტი (ფაილის სახელში `01-00` წერია).

➡️ **რა ვკითხოთ IT-ს:** „ექსპორტი ან view რომელი საათისთვის განახლდება? ჩვენ შემდეგ გავუშვებთ.“

Q35. 🟡 რატომ ასლი და არა ცოცხლად კითხვა?

🗣 **მოკლე პასუხი:** „განაწილება მილიონობით ჩანაწერს აანალიზებს: რომელია კოდის პირველი ოქმი, ვინ არის ამზომველი. ცოცხლად კითხვა თქვენს სისტემას დღისით დატვირთავდა, და თქვენი სისტემის მიუწვდომლობისას ჩვენც გავჩერდებოდით. ასლი ორივე პრობლემას ხსნის. ცოცხლად მხოლოდ მცირე, ხშირად ცვალებად მონაცემს წავიკითხავთ.“

🔧 **ტექნიკური ახსნა:**

- ✅ REGFIELDALL ≈ 2.33 მლნ ჩანაწერია;
- „პირველი ოქმის“ გამოთვლა correlated subquery-ებით ხდება (`source.query_candidates`). ეს ჩვენს ასლზე უნდა გაეშვას და არა წყაროზე.

💬 **მარტივად რომ გავიგოთ:** დიდი წიგნიდან ერთხელ გადავიწერთ საჭირო გვერდებს და მერე ჩვენს ასლს ვფურცლავთ. წიგნის პატრონს ყოველ წუთს არ ვაწუხებთ.

📌 **ჩვენი პროექტიდან:** ✅ Distributor-ის ასლი ≈ 1 GB; ✅ ძველი პროგრამაც ასე მუშაობს.

❓ **გაურკვეველია:** —

➡️ **რა ვკითხოთ IT-ს:** —

Q36. 🟢 ჩვენი სისტემა რომ მიუწვდომელი იყოს, თქვენი იმუშავებს?

🗣 **მოკლე პასუხი:** „კი. QC-ის მთელი სამუშაო ჩვენს ბაზაზეა: შემოწმება, პასუხები, სტატისტიკა. წყაროს გაჩერება მხოლოდ ახალ განაწილებას აყოვნებს, და ისიც ბოლო ჩატვირთულ მონაცემზე გაგრძელდება. იმპორტი ხელახლა ცდის და ადმინს ატყობინებს.“

🔧 **ტექნიკური ახსნა:**

- graceful degradation;

- ✅ retry `fetch_sheets.py`-ში;
- 💡 `import_runs` + განგაში;
- 💡 „ბოლო განახლება“ ეკრანზე.

🗨️ **მარტივად რომ გავიგოთ:** თუ ღამის „ამანათი“ არ მოვიდა, გუშინდელით ვაგრძელებთ.

📌 **ჩვენი პროექტიდან:** სახელმძღვანელო C.11.

❓ **გაურკვეველია:** —

➡️ **რა ვკითხოთ IT-ს:** „თქვენი სისტემების დაგეგმილ გათიშვებს როგორ აცხადებთ?“

Q37. 🟢 ჩვენს სისტემებში რამეს წერთ?

🗣️ **მოკლე პასუხი:** „არა. ეს პრინციპია, და კოდიც ამას იცავს: წყარო ფაილები მხოლოდ წასაკითხად იხსნება, Google ცხრილებს მხოლოდ GET მოთხოვნები ეგზავნება, და თქვენს ბაზებზე მხოლოდ `SELECT`-ის უფლებას ვითხოვთ.“

🔧 **ტექნიკური ახსნა:**

- ✅ `source.connect()`: `?mode=ro`;
- ✅ `fetch_sheets.py`: „ჩაწერის (POST) მოქმედებებს ეს სკრიპტი არ იძახებს — არც შეუძლია“;
- ⚠️ ძველი Distributor კონტროლის ცხრილში წერს. ახალი პლატფორმა მას ჩაანაცვლებს.

🗨️ **მარტივად რომ გავიგოთ:** მხოლოდ ვკითხულობთ და არასოდეს ვწერთ თქვენთან.

📌 **ჩვენი პროექტიდან:** ✅ README: „არსებულ სისტემებში პროტოტიპი არაფერს წერს“.

❓ **გაურკვეველია:** —

➡️ **რა ვკითხოთ IT-ს:** —

Q38. 🔴 როგორ ვიცით, რომ QC მონაცემი ზუსტია?

🗣️ **მოკლე პასუხი:** „სამი გზით:

- წესებს სისტემა იცავს: სავალდებულო ველები, ვინ რას პასუხობს, ერთი პასუხი ერთ კოდზე;
- ყოველი ციფრი ერთი ფუნქციით ითვლება ყველა როლისთვის, ამიტომ ციფრები ერთმანეთს ვერ დაშორდება;
- ძველ მონაცემზე შეჯერება გავაკეთეთ: ძველი შემაჯამებელი თითოეულ ოპერატორზე ზუსტად დაემთხვა, დოკუმენტირებული გამონაკლისებით.“

🔧 **ტექნიკური ახსნა:**

- ✅ `workflow.apply()` — guard-ები;

- ✅ `services/stats.py` — „ერთი წყარო, ერთი განმარტება“;
- ✅ შეჯერება (`ARCHITECTURE.md` §9): შემოწმებული ოპერატორებზე = ჩვენი + დუბლიკატები; 271 დავა = QC მენეჯერთან მყოფი სისტემური ოქმები;
- ✅ მეტრიკების ლექსიკონი (`ARCHITECTURE.md` §7);
- ⚠️ ცნობილი განსხვავებები ძველ პანელთან:
  - ATR-04-ის სიმძიმე (C2);
  - დაზუსტება დადებითად აღარ ითვლება;
  - მენეჯერზე მიკუთვნება ყველა მენეჯერზე ხდება.

💬 **მარტივად რომ გავიგოთ:** ციფრები ერთი „სასწორით“ იწონება ყველასთვის, და ძველ „სასწორთან“ შედარებით დავამოწმეთ.

📌 **ჩვენი პროექტიდან:** ✅ 44 538 → 44 159 + 379.

❓ **გაურკვეველია:** ❓ ძველი მონაცემი (სახელები, ცხრილის ხელით შეცდომები) ისევ ძველი მონაცემის ხარისხისაა.

➡️ **რა ვკითხოთ IT-ს:** —

Q39. 🟡 დღეს REGFIELDALL-ს როგორ იღებთ?

🗣️ **მოკლე პასუხი:** „ღამის GDB ექსპორტიდან, რომელიც ქსელურ საქაღალდეში დევს. განაწილების პროგრამა ბოლო ფაილს პოულობს და საჭირო ველებს ლოკალურ ასლში ტვირთავს, ახალი პლატფორმა კი ამ ასლს წასაკითხად ხსნის. ეს ჩემი კომპიუტერის ქსელური დისკით მუშაობს, ამიტომ სერვერისთვის ოფიციალური გზა გვჭირდება.“

🔧 **ტექნიკური ახსნა:**

- ✅ `Z:\REGFIELDALL\REGFIELDALL_<YYYY-MM-DD>_01-00_<N>.gdb` ;
- ფენა `NAPR_VEXP_FLD_GPARCELS_4GDB` ;
- GDAL;
- ≈ 2.33 მლნ ჩანაწერი, 40–60 წმ;
- ბოლო: `REGFIELDALL_2026-10-19_01-00_93.gdb` .

💬 **მარტივად რომ გავიგოთ:** ყოველ ღამე სადღაც ჩნდება ფაილი; ჩემს კომპიუტერს ის „თაროზე“ დევს. სერვერს ეს თარო ჯერ არ აქვს.

📌 **ჩვენი პროექტიდან:** ✅ `NAPR_Distributor/README.md` §1; ✅ `napr_data.sqlite` / `import_meta` .

❓ **გაურკვეველია:** ❓ ვინ ქმნის ექსპორტს, რომელი სისტემიდან, და რამდენ ხანს ინახება ძველი ფაილები.

➡️ **რა ვკითხოთ IT-ს:** „ეს ექსპორტი თქვენი სამსახური ქმნის? შეიძლება, ცალკე CSV-ად ან view-დაც გაკეთდეს?“

Q40. 🔴 Pre-QC-ს REDI-სთან როგორ აკავშირებთ? სერვერიდან REDI-ში შეხვალთ?

🗣 **მოკლე პასუხი:** „დღეს Pre-QC თანამშრომლის ბრაუზერის გაფართოებით მუშაობს: ადამიანი REDI-ში თავისი ანგარიშით შედის, გაფართოება გვერდს კითხულობს და იმავე კომპიუტერზე გაშვებულ ვალიდატორს გადასცემს. სერვერს ასეთი სესია არ აქვს, და მისი ხელოვნურად შექმნა არ გვინდა. ამიტომ სერვერზე Pre-QC-ისთვის ოფიციალური გზა გვჭირდება: ოქმის API ან ექსპორტი. ეს ღია კითხვაა, და პლატფორმის პირველ ეტაპს არ ეხება.“

🔧 **ტექნიკური ახსნა:**

- ✅ `app/extension/manifest.json`: `content_scripts` → `https://api.napr.gov.ge/lr/redi*`; `host_permissions` → `https://api.napr.gov.ge/*`, `http://127.0.0.1:8000/*`;
- ✅ REDI-ს გვერდები: `REDI.VIEW.PAGE`, `DYNAMIC.ABSTRACT.PAGE`, `REGCARD`; ფოტოები `bs.napr.gov.ge/GetBlob`;
- ✅ ვალიდატორი read-only-ა: „არაფერს ცვლის“;
- ⚠️ `app/README.md`: „REDI-ის შენახული ფაილები შეიცავს API გასაღებს და პერსონალურ მონაცემებს“;
- სერვერული ინტეგრაციის ვარიანტები:
  - **(ა)** REDI-ის ოფიციალური API სერვისის ანგარიშით;
  - **(ბ)** ოქმის ღამის ექსპორტი;
  - **(გ)** Validator ლოკალურად რჩება და შედეგს API-ით აგზავნის. ასეა Pre-QC hosted-ის დიზაინში, `POST /api/preqc/checks`.

💬 **მარტივად რომ გავიგოთ:** დღეს Pre-QC „ადამიანის თვალით“ ხედავს REDI-ს. სერვერს თვალები არ აქვს: მას „ოფიციალური ფანჯარა“ (API) სჭირდება, ან ვალიდატორი ისევ ადამიანთან რჩება და მხოლოდ შედეგს აგზავნის.

📌 **ჩვენი პროექტიდან:** ✅ პლატფორმაში Pre-QC დემოა (216 ჩანაწერი).

❓ **გაურკვეველია:** ❓ არსებობს თუ არა REDI-ს API; შეყვანის ფორმატის გადაწყვეტილება (`OPEN_QUESTIONS.md` #8).

➡️ **რა ვკითხოთ IT-ს:** „REDI-ს ოქმის მონაცემზე სისტემური (სერვისის) წვდომა არსებობს? თუ არა, რომელი ვარიანტი იქნებოდა თქვენთვის მისაღები: (ა), (ბ) თუ (გ)?“

Q41. 🟡 WFS რატომ გჭირდებათ და ვისი ანგარიშით?

🗣 **მოკლე პასუხი:** „WFS-ს Pre-QC იყენებს რუკაზე შემოწმებებისთვის: ნაკვეთის საზღვრები, გადაფარვა, მეზობლები. დღეს `.env` ფაილში შენახული მომხმარებლით მუშაობს. საწარმოოსთვის სერვისის ანგარიში გვჭირდება, მხოლოდ წაკითხვით, და სერვერის IP-ის დაშვება. პლატფორმის პირველ ეტაპს WFS არ სჭირდება.“

🔧 **ტექნიკური ახსნა:**

- ✅ `wfs.py`: `https://wblr.napr.gov.ge/data/SLR/ows`, `GetFeature`, WFS 2.0.0, `EPSG:32638`, `outputFormat=application/json`, `cql_filter`, `propertyName` (მინიმიზაცია), HTTP Basic;
- ✅ ქეში 300 წმ;
- ✅ 1 retry დაკავშირების შეცდომაზე; წაკითხვის timeout 90 წმ.

💬 **მარტივად რომ გავიგოთ:** რუკის სერვისს „ვეკითხებით“, ნაკვეთი სად არის და როგორია. ამისთვის „საშვი“ გვჭირდება, რომელიც ადამიანს კი არა, სისტემას ეკუთვნის.

📌 **ჩვენი პროექტიდან:** სახელმძღვანელო C.4.

❓ **გაურკვეველია:** ❓ ⚠️ ვისი ანგარიშია დღევანდელი WFS მომხმარებელი, და რა ნებართვებით.

➡️ **რა ვკითხოთ IT-ს:** „SLR WFS-ზე სერვისის ანგარიში შეიძლება? IP-ის შეზღუდვა აქვს?“

Q42. 🔴 Apps Script-ის საჯარო URL token-ით? ეს უსაფრთხოების რისკია.

🗣️ **მოკლე პასუხი:** „ვეთანხმებით, საწარმოოში ეს არ დარჩება. დღეს ორი ასეთი ხიდი მხოლოდ კითხვისთვის გამოიყენება: ძველი ცხრილის ერთჯერადი გადმოტანისთვის და Field Group-ისთვის. სკრიპტი მხოლოდ საჭირო ველებს აბრუნებს, პირადი ნომრების გარეშე. საწარმოოში გუნდებს ადმინი პლატფორმაში მართავს, ძველი ცხრილი კი მიგრაციის შემდეგ არქივი ხდება. token-ებს შევცვლით.“

🔧 **ტექნიკური ახსნა:**

- Web App: „Execute as: Me“, „Who has access: Anyone“, საერთო token query string-ში;
- რისკები:
  - token-ის გაჟონვა (ლოგები, ისტორია);
  - მფლობელზე დამოკიდებულება;
  - ✅ Field Group-ის სკრიპტი მინიმალურ ველებს აბრუნებს;
- ⚠️ token-ები სხვა პროექტების ფაილებში ღია ტექსტითაა შენახული;
- 💡 ალტერნატივა, თუ Sheets-ის წაკითხვა მაინც დაგვჭირდება: Google service account, `spreadsheets.readonly`, ცხრილის Viewer-ად გაზიარება (ასეა Pre-QC hosted-ის დიზაინში).

💬 **მარტივად რომ გავიგოთ:** დღეს ეს „დროებითი ხიდია“, პაროლიანი ბმულით. ახალ სისტემაში ხიდი აღარ დაგვჭირდება.

📌 **ჩვენი პროექტიდან:** ✅ `scripts/fetch_sheets.py`: „token-ები არსად იბეჭდება და data/raw-ში არ ინახება“.

❓ **გაურკვეველია:** —

➡️ **რა ვკითხოთ IT-ს:** „ძველი ცხრილის საბოლოო ასლს cutover-ის დღეს როგორ შევინახავთ? Google Workspace-ის ადმინი თქვენთან არის?“

Q43. 🟡 სპორადულის ფაილი დღეს მეილით მოდის, პირადი მონაცემებით. როგორ ამუშავებთ?

🗣️ **მოკლე პასუხი:** „ფაილიდან მხოლოდ ათ საჭირო სვეტს ვკითხულობთ: განმცხადებლების პირადი ნომრები, ტელეფონები და მისამართები საერთოდ არ იკითხება, და ატვირთული ფაილი დამუშავებისთანავე იშლება. მაგრამ თავად მეილით გაგზავნა რისკია. ამიტომ გთხოვთ, ფაილი view-ით ან SFTP-ით მოგვაწოდოთ, და მხოლოდ საჭირო სვეტებით.“

🔧 **ტექნიკური ახსნა:**

- ✅ `sporadic.read_file`: `NEEDED`, `openpyxl` `read_only`;
- APPLICS, APP_PERSONS, REG_EMP და ADDRESS არ იკითხება;
- ✅ ფაილი იშლება;
- ✅ 20.10-ის ფაილი: 50 320 ჩანაწერი;
- ✅ `FLD_PAPER_ID` ძველი სპორადულის 2 908 ჩანაწერიდან 2 907-ს ემთხვევა.

💬 **მარტივად რომ გავიგოთ:** ამანათს ვხსნით, საჭირო ორ ფურცელს ვიღებთ, დანარჩენს მაშინვე ვანადგურებთ. უკეთესი იქნება, თუ ამანათში მხოლოდ ეს ორი ფურცელი ჩაიდება.

📌 **ჩვენი პროექტიდან:** ✅ `services/sporadic.py`, docstring.

❓ **გაურკვეველია:** ❓ ფილტრის წესები (Q11 ა–დ).

➡️ **რა ვკითხოთ IT-ს:** „ამ სიის ექსპორტი საიდან კეთდება? იმავე ფორმატით, მხოლოდ ამ სვეტებით, view-დ ან ღამის ფაილად შეიძლება?“

ე. უსაფრთხოება

Q44. 🟢 მომხმარებლები როგორ შევლენ?

🗣️ **მოკლე პასუხი:** „Google ანგარიშით. პაროლს ჩვენ ვერც ვხედავთ და ვერც ვინახავთ. შესვლა Google-ის გვერდზე ხდება, Google კი გვიდასტურებს ელფოსტას. სისტემაში შედის მხოლოდ ის, ვისი ელფოსტაც ჩვენმა ადმინმა წინასწარ დაარეგისტრირა, და როლსაც ადმინი ანიჭებს.“

🔧 **ტექნიკური ახსნა:**

- ✅ OpenID Connect, Authorization Code + PKCE (S256), `state`, `nonce`, `prompt=select_account`;
- ✅ token endpoint სერვერიდან, TLS-ით;
- ✅ მოწმდება `iss`, `aud`, `exp`, `nonce`, `email_verified` და, არჩევით, `hd`;
- ✅ შემდეგ `users.email` (`active`) → სერვერის სესია;
- ✅ scope: `openid email profile`;

- ✅ უცნობი ელფოსტა → `login_denied` აუდიტში.

💬 **მარტივად რომ გავიგოთ:** Google „ვინაობას ადასტურებს“, ჩვენ კი „ბეჯს ვაძლევთ“. Google-ის ანგარიში მარტო საკმარისი არ არის: ჩვენს სიაში უნდა იყო.

📌 **ჩვენი პროექტიდან:** ✅ `app/routes/google_auth.py`; ✅ ტესტი `test_google_login_only_for_registered_email`. ⚠️ ბაზაში 0 ელფოსტაა.

❓ **გაურკვეველია:** ❓ Workspace თუ პირადი Gmail (Q46).

➡️ **რა ვკითხოთ IT-ს:** „OAuth client-ის შექმნას სააგენტოს Google Cloud-ში თქვენ გააკეთებთ?“

Q45. 🔴 რატომ Google და არა ჩვენი Active Directory / LDAP / Microsoft Entra?

🗣️ **მოკლე პასუხი:** „ძირითადი მიზეზი ველია: ≈ 250 ამზომველი ტელეფონით, სააგენტოს ქსელის გარეთ მუშაობს, და, ჩვენი ინფორმაციით, ბევრს სააგენტოს დომენის ანგარიში არ აქვს. Google ანგარიში ყველას აქვს. თუ ყველა თანამშრომელს AD ანგარიში აქვს და გარედანაც უსაფრთხოდ ხელმისაწვდომია, Entra ID ან სხვა OIDC პროვაიდერი ტექნიკურად მარტივად ჩაირთვება: OIDC-ის იგივე სტანდარტია.“

🔧 **ტექნიკური ახსნა:**

- ჩვენი შესვლა სტანდარტულ OIDC-ს იყენებს: Authorization Code + PKCE;
- Entra ID (Azure AD), Keycloak, ADFS (OIDC რეჟიმი) იმავე ნაკადს უჭერს მხარს;
- შეიცვლება endpoint-ები, `iss` და claim-ები (`email` / `upn` / `preferred_username`);
- LDAP-ით პირდაპირი შესვლა (პაროლი ჩვენს ფორმაში) ნაკლებად უსაფრთხოა: პაროლი ჩვენს სერვერზე გადის.

💬 **მარტივად რომ გავიგოთ:** „პირადობის შემმოწმებელი“ შეიძლება იყოს Google ან სააგენტოს საკუთარი სისტემა. ჩვენ ორივესთან შეგვიძლია მუშაობა. Google იმიტომ ავირჩიეთ, რომ ველში ყველას აქვს.

📌 **ჩვენი პროექტიდან:** ✅ `google_auth.py` — endpoint-ები კონსტანტებია (`AUTH_URL`, `TOKEN_URL`, `ISSUERS`). მეორე პროვაიდერის დამატება მცირე ცვლილებაა 💡.

❓ **გაურკვეველია:** ❓ ⚠️ აქვს თუ არა ყველა ამზომველს სააგენტოს ანგარიში. ეს ჩვენი ვარაუდია და არა ფაქტი.

➡️ **რა ვკითხოთ IT-ს:** „ამზომველებს სააგენტოს დომენის ანგარიში აქვთ? თუ კი, Entra/AD-ის OIDC ვარიანტი გვირჩევნიათ?“

Q46. 🔴 Workspace არ გვაქვს. პირადი Gmail-ით შესვლა უსაფრთხოა?

🗣️ **მოკლე პასუხი:** „Google-ის ანგარიშის დაცვა (2FA და საეჭვო შესვლის აღმოჩენა) პირად Gmail-საც აქვს. ჩვენ მხოლოდ დარეგისტრირებულ ელფოსტას ვუშვებთ. რისკი ისაა, რომ ანგარიშს თანამშრომელი ფლობს და არა სააგენტო. ამიტომ წასვლისას წვდომას ჩვენ

ვთიშავთ, ერთი ღილაკით და მყისიერად. 2FA-ს სავალდებულოდ ვერ ვაქცევთ, მაგრამ ვურჩევთ.“

🔧 **ტექნიკური ახსნა:**

- External OAuth client;
- ✅ ავტორიზაცია ჩვენს `users` სიაზეა და არა Google-ის დომენზე;
- ✅ `active = false` → სესია მყისიერად უქმდება, შესვლა კი უარყოფილია;
- 💡 დამატებითი ზომები:
  - ადმინის, QC მენეჯერის და მენეჯერის როლებისთვის Workspace ან AD ანგარიში, თუ არსებობს;
  - შესვლის ჟურნალის მონიტორინგი;
  - სესიის ვადის შემცირება.

💬 **მარტივად რომ გავიგოთ:** კარის გასაღები (Gmail) ადამიანისაა, მაგრამ კარის საკეტი (ჩვენი სია) ჩვენია. ადამიანი წავიდა, ჩვენ საკეტს ვცვლით.

📌 **ჩვენი პროექტიდან:** ✅ `IT_REQUEST.md` §6: „თანამშრომლები პირადი Gmail-ით (მენეჯერები აგროვებენ სახელს, გვარს და ელფოსტას)“.

❓ **გაურკვეველია:** ❓ სააგენტოს პოლიტიკა პირადი ანგარიშების გამოყენებაზე.

➡️ **რა ვკითხოთ IT-ს:** „პირადი Gmail-ით შესვლა თქვენი პოლიტიკით დასაშვებია? თუ არა, რა ანგარიში აქვს ყველა თანამშრომელს?“

Q47. 🟢 როლებს ვინ ანიჭებს? RBAC როგორ მუშაობს?

🗣 **მოკლე პასუხი:** „შვიდი როლია. მათ ჩვენი ორი ადმინი ანიჭებს, ადმინისტრირების გვერდიდან. ყოველი ცვლილება აუდიტში იწერება: ვინ, როდის, რა იყო და რა გახდა. უფლებები სერვერზე სამ დონეზე მოწმდება: გვერდი, ჩანაწერი და მოქმედება.“

🔧 **ტექნიკური ახსნა:**

- ✅ `security.require()` — მარშრუტი;
- ✅ `scope.item_filter` / `can_view` — ჩანაწერი;
- ✅ `workflow.available_actions` — მოქმედება და მდგომარეობა;
- ✅ აქტიური როლი სესიაშია; როლის ცვლა (`POST /role`) მოწმდება და აუდიტში იწერება;
- ✅ ადმინი საკუთარ ადმინის როლს ვერ მოიხსნის.

💬 **მარტივად რომ გავიგოთ:** ბეჯი გვეუბნება, რომელ ოთახში შედიხარ. ოთახში კი მხოლოდ შენი საქაღალდეები ჩანს.

📌 **ჩვენი პროექტიდან:** როლების მატრიცა — Handoff §4.5.

❓ **გაურკვეველია:** ❓ კოორდინატორისა და სამსახურის უფროსის როლები ჯერ დემოა.

➡️ **რა ვკითხოთ IT-ს: —**

Q48. 🟢 როგორ ვიცით, რომ ოპერატორი სხვის ოქმს ვერ ნახავს?

🗣️ **მოკლე პასუხი:** „ყოველი სია და ბარათი სერვერზე იფილტრება, აქტიური როლისა და საკუთრების მიხედვით. თუ ვინმე მისამართში სხვა ნომერს აკრეფს, სერვერი პასუხობს „ვერ მოიძებნა“, და ჩანაწერის არსებობასაც არ ადასტურებს. ეს ავტომატური ტესტებით მოწმდება.“

🔧 **ტექნიკური ახსნა:**

- ✅ IDOR-ისგან დაცვა: `items._load()` → `scope.can_view()` → 404;
- ✅ ფაილებზეც (`/attachments/{id}`) და API-ზეც (`/api/items/{id}`) იგივე;
- ✅ სიები SQL-შივე იფილტრება (`items_query`);
- ✅ ტესტები: `test_operator_sees_only_own` , `test_field_sees_nothing_before_send_and_nothing_positive` , `test_field_does_not_see_qc_internal_history` .

💬 **მარტივად რომ გავიგოთ:** „4513“-ის აკრეფა კარს ვერ გააღებს, რადგან სერვერი ყოველ ჯერზე ამოწმებს, შენია თუ არა.

📌 **ჩვენი პროექტიდან:** ✅ `app/scope.py` , docstring.

❓ **გაურკვეველია:** ❓ დამოუკიდებელი პენტესტი არ ჩატარებულა.

➡️ **რა ვკითხოთ IT-ს:** „საწარმოომდე უსაფრთხოების შემოწმება (პენტესტი) გაქვთ სტანდარტად?“

Q49. 🟡 თანამშრომელი რომ სხვა გუნდში გადავიდეს?

🗣️ **მოკლე პასუხი:** „ადმინი გუნდს უცვლის: ძველი წევრობა თარიღით იხურება და ახალი იხსნება. ახალი ოქმები ახალ მენეჯერთან მივა. უკვე განაწილებული ოქმები იმ მენეჯერთან რჩება, ვისაც განაწილების დროს ეკუთვნოდა. საჭიროებისას QC მენეჯერს ოქმზე მენეჯერის ხელით შეცვლა შეუძლია. ამზომველი საკუთარ ოქმებს ყოველთვის ხედავს.“

🔧 **ტექნიკური ახსნა:**

- ✅ `team_members.valid_from` / `valid_to` (`routes/admin.user_save`);
- ✅ `services/seed.manager_map` — მიმდინარე წევრობა;
- ✅ `item_people` და `field_manager_id` განაწილებისას ფიქსირდება;
- ✅ `workflow` : `assign_manager` ;
- ❓ ბიზნეს-კითხვა: ძველი ღია ოქმები ახალ მენეჯერს უნდა გადაეცეს ავტომატურად? ჯერ არ გადაწყვეტილა. 💡 მასობრივი გადაწერის ინსტრუმენტი.

💬 **მარტივად რომ გავიგოთ:** ადამიანი გადავიდა ახალ განყოფილებაში: ახალი სამუშაო ახალ უფროსთან მიდის, ძველი საქმე კი ძველ უფროსთან სრულდება, თუ სხვა გადაწყვეტილება არ მიიღეს.

📌 **ჩვენი პროექტიდან:** სახელმძღვანელო D.13.

❓ **გაურკვეველია:** ❓ ხელმძღვანელობის წესი ძველ ღია ოქმებზე.

➡️ **რა ვკითხოთ IT-ს:** „გუნდის ცვლილებას HR სისტემაში აღრიცხავთ? მისი წაკითხვა შეიძლება, რომ ხელით არ ვაკეთოთ?“

Q50. 🟡 თანამშრომელი რომ წავიდეს?

🗣️ **მოკლე პასუხი:** „ადმინი მას თიშავს. სესია მაშინვე უქმდება, და Google-ით შესვლაზე უარს მიიღებს. მისი ისტორია და ოქმები რჩება, რადგან ის ნამდვილად მუშაობდა. ჩვენ ვინმე უნდა გვაცნობოს, ამიტომ HR-იდან შეტყობინების პროცესი გვჭირდება.“

🔧 **ტექნიკური ახსნა:**

- ✅ `users.active = false` → `read_session()` ყოველ მოთხოვნაზე ამოწმებს;
- ✅ `google_callback` მხოლოდ `active`-ს ეძებს;
- ⚠️ G18: გათიშვა გუნდის წევრობას არ ხურავს, ამიტომ ახალ განაწილებაში ოქმები ისევ ძველ მენეჯერს მიეწერება, სანამ წევრობა არ დაიხურება. 💡 გასასწორებელია;
- ✅ წასული მენეჯერის ამზომველების ოქმები „დაუჯგუფებელში“ ხვდება; QC მენეჯერი ხელით მიუწერს;
- 💡 წასული ოპერატორის ოქმების მასობრივი გადანაწილება.

💬 **მარტივად რომ გავიგოთ:** ბეჯს ვართმევთ მაშინვე, მაგრამ მისი ხელმოწერები ძველ საბუთებზე რჩება.

📌 **ჩვენი პროექტიდან:** ✅ `app/security.py`, `read_session`.

❓ **გაურკვეველია:** ❓ HR-ის შეტყობინების პროცესი.

➡️ **რა ვკითხოთ IT-ს:** „ახალი და წასული თანამშრომლების შესახებ IT-ს როგორ ეცნობება? იმავე ნაკადში ჩვენი ადმინებიც ჩავრთოთ?“

Q51. 🟢 პაროლები და საიდუმლოებები სად ინახება?

🗣️ **მოკლე პასუხი:** „კოდში არცერთი არ წერია. ყველაფერი სერვერის გარემოს ცვლადებიდან ან თქვენი საიდუმლოების საცავიდან იკითხება. client secret-ი და ბაზის პაროლი პირდაპირ სერვერზე ჩაწერეთ; ჩვენ მათი ნახვაც არ გვჭირდება.“

🔧 **ტექნიკური ახსნა:**

- ✅ `app/config.py`: `QCP_DATABASE_URL`, `QCP_SESSION_SECRET`, `QCP_GOOGLE_CLIENT_SECRET`, `QCP_SMTP_PASSWORD`;

- ⚠️ თუ `QCP_SESSION_SECRET` არ არის მითითებული, ის `data/.session_secret` ფაილში იქმნება. 💡 საწარმოოში ცვლადით;
- ✅ token-ები არ იბეჭდება;
- 💡 ძველი პროექტების token-ების როტაცია.

💬 **მარტივად რომ გავიგოთ:** გასაღებები სეიფშია და არა სამუშაო მაგიდაზე.

📌 **ჩვენი პროექტიდან:** ✅ `.gitignore`: `.env`, `data/`.

❓ **გაურკვეველია:** ❓ IT-ის საიდუმლოების საცავი.

➡️ **რა ვკითხოთ IT-ს:** „საიდუმლოების მართვის სტანდარტული ინსტრუმენტი გაქვთ (Vault და სხვ.)?“

Q52. 🟢 აუდიტი გაქვთ?

🗣️ **მოკლე პასუხი:** „ორ დონეზე. ყოველი ოქმის სრული ისტორია: ვინ, რა როლით, რა გააკეთა, საიდან სად, როდის, კომენტარით. ეს მხოლოდ ემატება. ადმინისტრაციული მოქმედებების ჟურნალი: შესვლა, უარყოფილი შესვლა, როლის ცვლა, მომხმარებლის ცვლილება „ადრე / შემდეგ“, კოდები, პარამეტრები.“

🔧 **ტექნიკური ახსნა:**

- ✅ `qc_events` — append-only, კოდი არ ცვლის. 💡 ბაზის დონეზეც: `REVOKE UPDATE, DELETE`;
- ✅ `audit_log`: `actor_id, action, entity, entity_id, detail (JSON), ip, at`;
- ⚠️ G4: IP proxy-ს უკან.

💬 **მარტივად რომ გავიგოთ:** „ვინ, რა, როდის“ — წიგნი, რომელშიც მხოლოდ იწერება.

📌 **ჩვენი პროექტიდან:** ✅ ადმინისტრირება → აუდიტი; ბარათზე — ისტორია.

❓ **გაურკვეველია:** —

➡️ **რა ვკითხოთ IT-ს:** „აუდიტის ჩანაწერები თქვენს ცენტრალურ SIEM-შიც უნდა გადავიდეს?“

Q53. 🔴 პერსონალური მონაცემის დაცვა და შესაბამისობა (compliance)?

🗣️ **მოკლე პასუხი:** „მინიმიზაციის პრინციპით ვმუშაობთ: მოქალაქეების პირად ნომრებსა და ტელეფონებს არ ვკითხულობთ. თანამშრომლის პირადი ნომერი ინახება მხოლოდ ვინაობის შესადარებლად, ნიღბით ჩანს და ლოგში არ იწერება. ყველაფერი HTTPS-ით და როლებით არის დაცული. სამართლებრივი საფუძველი და შენახვის ვადა თქვენს და იურიდიულ სამსახურთან უნდა შევათანხმოთ. მონაცემის აღწერას მოგაწვდით.“

🔧 **ტექნიკური ახსნა:**

- ✅ `users.personal_id` (🔐, `__repr__`-ში არ იბეჭდება);
- ✅ სპორადულის ფაილის სვეტების ფილტრი;
- ✅ `fetch_sheets.py`-ში SKIP_TABS = „პირადი ნომრები“;
- ❓ QC კომენტარებში შეიძლება მოქალაქეების სახელები იყოს;
- 💡 დამატებითი ზომები:
  - დისკის ან ბაზის დაშიფვრა;
  - დაშიფრული ბექაპი;
  - ექსპორტის შეზღუდვა როლით;
  - ლოგების გასუფთავება (9+ ციფრი → `[ID]`, როგორც Pre-QC hosted-ში).

💬 **მარტივად რომ გავიგოთ:** რაც არ გვჭირდება, არ ვიღებთ. რაც გვჭირდება, ვმალავთ და ვიცავთ. წესს კი კანონი და იურისტები ამბობენ.

📌 **ჩვენი პროექტიდან:** ✅ README, „მონაცემის დაცვა“.

❓ **გაურკვეველია:** ❓ ⚠️ პირადი ნომრის შენახვის სამართლებრივი საფუძველი; ❓ სააგენტოს უსაფრთხოების სტანდარტი (მაგ. ISO 27001-ის მოთხოვნები).

➡️ **რა ვკითხოთ IT-ს:** „ახალი სისტემის პერსონალური მონაცემის შეფასებას (DPIA) ვინ ატარებს სააგენტოში? რა ფორმა გაქვთ?“

Q54. 🔴 OWASP-ის ტიპური სისუსტეები: XSS, SQL injection, ფაილის ატვირთვა?

🗣️ **მოკლე პასუხი:** „SQL შეკითხვები პარამეტრებით იგზავნება ORM-ის მეშვეობით. HTML-ში მონაცემს Jinja2 ავტომატურად აქცევს უსაფრთხოდ (escape). ფაილის ატვირთვა მხოლოდ კონტროლს შეუძლია: სურათი ან PDF, 10 MB-მდე, შემთხვევითი სახელით, და გაცემისას წვდომა მოწმდება. CSRF-ზე Origin-ის შემოწმება და SameSite cookie გვაქვს, CSRF token-ს საწარმოომდე დავამატებთ. პენტესტი სასურველია.“

🔧 **ტექნიკური ახსნა:**

- ✅ SQL injection: SQLAlchemy-ის პარამეტრები; `source.py`-ში `?` placeholder-ები;
- ✅ XSS: Jinja2 autoescape (FastAPI-ის `Jinja2Templates` `.html` შაბლონებზე ნაგულისხმევად ჩართულია); `HttpOnly` cookie; მეილის ნახვის გვერდს CSP აქვს;
- 💡 მთელ საიტზე CSP;
- ✅ ატვირთვა: `ALLOWED_UPLOAD_TYPES`, `MAX_UPLOAD_MB`, `uuid4`, `nosniff`. ⚠️ ტიპი `content_type`-ით მოწმდება და არა შიგთავსით;
- ⚠️ G8: CSRF token არ არის;
- ⚠️ G9: `/docs` ღიაა;
- 💡 rate limiting შესვლაზე (PIN-ზე `sleep(1)` არის, მაგრამ საწარმოოში დემო გათიშულია).

💬 **მარტივად რომ გავიგოთ:** ცნობილ „ჩამკეტებს“ ვიყენებთ. ზოგი ჯერ ბოლომდე არ არის დამონტაჟებული, და ესეც ვიცით.

📌 **ჩვენი პროექტიდან:** ✅ `app/main.py` — უსაფრთხოების თავსართები; ✅ `workflow.save_files`.

❓ **გაურკვეველია:** ❓ დამოუკიდებელი შემოწმება არ ყოფილა.

➡️ **რა ვკითხოთ IT-ს:** „უსაფრთხოების შემოწმების სიას (checklist) მოგვცემთ, რომ წინასწარ გავიაროთ?“

---

3. მიგრაცია, ოპერაციები, მხარდაჭერა

Q55. 🟡 ძველი მონაცემი როგორ გადმოვა?

🗣 **მოკლე პასუხი:** „პროტოტიპში უკვე გადმოვიტანეთ: 44 538 მწკრივი, 379 დუბლიკატი მოცილდა, 44 159 შემთხვევა. შეჯერება თითოეულ ოპერატორზე ზუსტად დაემთხვა ძველ შემაჯამებელს. საწარმოო მიგრაცია ცალკე უნდა დამტკიცდეს: ჯერ სატესტოზე, შეჯერებით, და მერე, შეთანხმებულ დღეს, ძველი ცხრილის დაკეტვით.“

🔧 **ტექნიკური ახსნა:**

- ✅ ETL: Apps Script GET → `data/raw/*.json` → `legacy_import`;
- ✅ მდგომარეობის წესები (Q/S/O);
- ✅ ალიასები;
- ✅ `legacy_extra` (ნედლი) და `approx` დროები;
- ✅ 1 550 ოქმს ამზომველი REGFIELDALL-იდან შეევსო;
- 💡 ნაბიჯები — სახელმძღვანელო F.9.

💬 **მარტივად რომ გავიგოთ:** „გადასვლა“ უკვე ვცადეთ, და ყველაფერი ითვლება. ნამდვილი გადასვლა ერთ დღეს მოხდება, წინასწარ რეპეტიციით.

📌 **ჩვენი პროექტიდან:** ✅ Handoff §4.9.

❓ **გაურკვეველია:** ❓ ძველი ისტორია ვერ აღდგება, რადგან ცხრილი სტატუსს ზედ წერდა.

➡️ **რა ვკითხოთ IT-ს:** —

Q56. 🟡 ძველი სისტემები პარალელურად იმუშავებს?

🗣 **მოკლე პასუხი:** „მოკლე პარალელურ პერიოდს ვთავაზობთ, ოღონდ ორმხრივი სინქრონის გარეშე: სამუშაო ერთ სისტემაში ხდება. პილოტის ჯგუფი ახალზე მუშაობს, დანარჩენები ძველზე. cutover-ის შემდეგ ძველი ცხრილი მხოლოდ საკითხავი ხდება, განაწილების პროგრამა კი ითიშება.“

🔧 **ტექნიკური ახსნა:** dual-write რისკიანია (კონფლიქტები). ⚠️ ერთი ოქმი ორ სისტემაში ერთდროულად არ უნდა განაწილდეს. ცხრილი — სახელმძღვანელო F.7.

💬 **მარტივად რომ გავიგოთ:** ორ რვეულში ერთდროულად არ ვწერთ. ერთი სამუშაოა, მეორე შესადარებელი.

📌 **ჩვენი პროექტიდან:** ✅ `DEMO.md`: „Google ცხრილები... პარალელური პერიოდის შემდეგ, როცა ციფრები დაემთხვევა“.

❓ **გაურკვეველია:** ❓ ხანგრძლივობა და პილოტის ჯგუფი.

➡️ **რა ვკითხოთ IT-ს:** —

Q57. 🔴 ახალი სისტემა რომ ვერ გამართლდეს — დაბრუნების გეგმა?

🗣️ **მოკლე პასუხი:** „პარალელური პერიოდის დროს ძველი ცხრილი ადგილზეა, ამიტომ დაბრუნება შესაძლებელია. cutover-ის შემდეგ დაბრუნებისთვის ახალი სისტემიდან ძველ ფორმატში ექსპორტი გვაქვს (Excel), რომ სამუშაო არ დაიკარგოს. ტექნიკური განახლების უკან დაბრუნებისთვის ბექაპი და მიგრაციის downgrade-ი გვაქვს.“

🔧 **ტექნიკური ახსნა:**

- ✅ სტატისტიკის Excel ექსპორტი (`/stats/export.xlsx`), ✅ განაწილების ექსპორტი (`/distribution/export/{token}`);
- 💡 ღია შემთხვევების ექსპორტი ძველი ცხრილის სვეტებით. ეს ჯერ არ არის — დასამატებელია;
- 💡 Alembic `downgrade`; ბექაპი deployment-მდე.

💬 **მარტივად რომ გავიგოთ:** ძველ სახლს მაშინვე არ ვყიდით. სანამ ახალში არ დავრწმუნდებით, გასაღები გვაქვს.

📌 **ჩვენი პროექტიდან:** —

❓ **გაურკვეველია:** ❓ „წარუმატებლობის“ კრიტერიუმები. ხელმძღვანელობასთან უნდა შეთანხმდეს.

➡️ **რა ვკითხოთ IT-ს:** —

Q58. 🟢 ხარისხს როგორ ამოწმებთ? ტესტები?

🗣️ **მოკლე პასუხი:** „57 ავტომატური ტესტი გვაქვს: სამუშაო პროცესი, ხილვადობა, განაწილება, ძველი მონაცემის წესები, სპორადული, მეილი, შესვლა და გვერდები ყველა როლით. ტესტები დროებით ბაზაზე მუშაობს და რეალურ მონაცემს არ ეხება. ამას ემატება შემოწმების სკრიპტები გაშვებულ სერვერზე.“

🔧 **ტექნიკური ახსნა:**

- ✅ `pytest -q` → 57 passed (≈ 6 წმ);
- ✅ `smoke_pages.py`, `smoke_stats.py`, `smoke_distribution.py`;
- ❓ არ არის: დატვირთვის ტესტი, PostgreSQL-ზე ტესტი, browser E2E;
- 💡 CI: ტესტები ყოველ ცვლილებაზე, git-ის შემდეგ.

💬 **მარტივად რომ გავიგოთ:** ყოველი ცვლილების შემდეგ „რობოტი" 57 სცენარს ამოწმებს. თუ რომელიმე გატყდა, ვხედავთ.

📌 **ჩვენი პროექტიდან:** ✅ `tests/`.

❓ **გაურკვეველია:** —

➡️ **რა ვკითხოთ IT-ს:** „CI/CD ინფრასტრუქტურა გაქვთ, სადაც ტესტები ავტომატურად გაეშვება?"

Q59. 🟡 განახლებას ვინ აყენებს და როგორ კონტროლდება ცვლილებები?

🗣 **მოკლე პასუხი:** „გვინდა წერილობითი პროცედურა: ვერსია ჩვენ ვამზადებთ, ტესტებით, ჯერ სატესტო გარემოზე. საწარმოოში თქვენი წესით ვაყენებთ: ან თქვენ, ან ჩვენ თქვენთან ერთად. ყოველ განახლებას ვერსიის ნომერი და ცვლილებების აღწერა აქვს. ვერსიის კონტროლის სისტემას ახლა ვაწყობთ."

🔧 **ტექნიკური ახსნა:**

- ⚠️ G17: git ჯერ არ არის;
- 💡 git tag → CHANGELOG → ტესტი → ბექაპი → `alembic upgrade` → restart → smoke → rollback გეგმა;
- ✅ ვერსია `app/__init__.py`-შია (`0.2.0`).

💬 **მარტივად რომ გავიგოთ:** თითო განახლებას ნომერი და „რა შეიცვალა" აქვს, და ჯერ „რეპეტიციაზე" მოწმდება.

📌 **ჩვენი პროექტიდან:** —

❓ **გაურკვეველია:** ❓ IT-ის change management.

➡️ **რა ვკითხოთ IT-ს:** „სერვერზე ჩვენი წვდომა (SSH/RDP) იქნება, თუ განახლებას მხოლოდ თქვენ აყენებთ?"

Q60. 🟢 მეილს როგორ გაგზავნით? სპამში არ მოხვდება?

🗣 **მოკლე პასუხი:** „თქვენი SMTP relay-ით, სააგენტოს გამგზავნი მისამართით, რომელსაც SPF და DKIM აქვს. ასე სპამში არ მოხვდება. მოცულობა მცირეა: დღეში ათეულობით შეტყობინება, ძირითადად საველე მენეჯერებთან. გაიგზავნება მხოლოდ მაშინ, როცა კონტროლი ✉-ს მონიშნავს. SMTP-მდე მეილები სისტემის „გამავალ ყუთში" ინახება."

🔧 **ტექნიკური ახსნა:**

- ✅ `services/notify.py`: `smtplib` + STARTTLS; სტატუსები `preview / sent / failed / no_address`;
- ✅ ადრესატი პასუხისმგებელი მენეჯერია, CC — დანარჩენი;
- ✅ HTML + ტექსტური ვერსია;
- ✅ ბმული ოქმზე, შინაარსი მინიმალური;
- ⚠️ G7: გაგზავნა მოთხოვნის შიგნით ხდება — 💡 outbox + worker.

💬 **მარტივად რომ გავიგოთ:** სააგენტოს „საფოსტო ყუთიდან“ ვაგზავნით, რომ ყველა ფოსტამ ნდობით მიიღოს.

📌 **ჩვენი პროექტიდან:** ✅ ადმინისტრირება → მეილები (preview); ⚠️ 0 ელფოსტა → ყველა `no_address`.

❓ **გაურკვეველია:** ❓ ავტომატური მეილი მონიშვნის გარეშე / დღის შეჯამება (`OPEN_QUESTIONS.md` #10).

➡️ **რა ვკითხოთ IT-ს:** „SMTP relay-ის მისამართი, პორტი და ავთენტიფიკაციის წესი? გამგზავნი მისამართის შექმნა (მაგ. `qc-noreply@…`) ვის ევალება?“

Q61. 🔴 რატომ არა ღრუბელში (Render, Azure...)? ძველი პანელი ხომ იქ იყო?

🗣️ **მოკლე პასუხი:** „ძველი სტატისტიკის პანელის ღრუბელში განთავსება ექსპერიმენტი იყო. ახალი პლატფორმა სააგენტოს სერვერზე იგეგმება: მონაცემი შიდაა, და წყაროებთან შიდა ქსელით გვიწევს კავშირი. თუ სააგენტოს დამტკიცებული ღრუბელი აქვს, ტექნიკურად იქაც იმუშავებს. ეს თქვენი და ხელმძღვანელობის გადაწყვეტილებაა.“

🔧 **ტექნიკური ახსნა:**

- ✅ `qc-dashboard-main/DEPLOY.md`: GitHub (private) + Render (Free) + Neon PostgreSQL, Frankfurt;
- Render-ის უფასო გეგმის პრობლემები: 15 წუთზე „ძილი“, ~50 წმ გაღვიძება, დროებითი დისკი;
- ✅ ახალი პლატფორმა ღრუბელზე არ არის მიბმული (`QCP_*`);
- ❓ ⚠️ ძველი პანელის ამჟამინდელი მდგომარეობა ღრუბელში გადასამოწმებელია, და, თუ აქტიურია, მისი გათიშვა ან ჩანაცვლება უნდა გადაწყდეს.

💬 **მარტივად რომ გავიგოთ:** ნაქირავებ ოფისში ვცადეთ. ახლა სააგენტოს შენობაში გადავდივართ, სადაც მონაცემი უფრო დაცულია და ახლოსაა.

📌 **ჩვენი პროექტიდან:** ✅ `OPEN_QUESTIONS.md` #5.

❓ **გაურკვეველია:** ❓ ძველი პანელის მდგომარეობა.

➡️ **რა ვკითხოთ IT-ს:** „სააგენტოს ღრუბლის პოლიტიკა გაქვთ? თანამშრომლების და QC მონაცემის გარე ღრუბელში შენახვა დასაშვებია?“

Q62. 🔴 დემო შესვლა (სიიდან ადამიანის არჩევა) საწარმოოში ხომ არ დარჩება?

🗣️ **მოკლე პასუხი:** „არა. საწარმოოში დემო შესვლა ითიშება, და შესვლა მხოლოდ Google-ით იქნება. დღეს ის ერთი პარამეტრით ირთვება და ითიშება. ნაგულისხმევად ჩართულია, ამიტომ საწარმოოსთვის ისე გადავაკეთებთ, რომ ის ფიზიკურად ვერ ჩაირთოს. ქსელში ჩვენებისას დღესაც PIN-ით არის დაცული.“

🔧 **ტექნიკური ახსნა:**

- ✅ `QCP_DEMO_LOGIN` (ნაგულისხმევი `1`);
- ✅ `QCP_DEMO_PIN` (`run_lan.bat` ყოველ გაშვებაზე ახალს ქმნის; PIN-ის გარეშე სიაც არ ჩანს; არასწორ PIN-ზე `sleep(1)`);
- ⚠️ G3: ნაგულისხმევი `1`, დემო cookie-ზე `Secure` არ არის. 💡 `QCP_ENV=production` → დემო შეუძლებელია, ან ნაგულისხმევი `0`.

💬 **მარტივად რომ გავიგოთ:** დემო „სტუმრის შესასვლელია“ ჩვენებისთვის. საწარმოოში ეს კარი სამუდამოდ იკეტება.

📌 **ჩვენი პროექტიდან:** ✅ `app/config.py`, `routes/auth.py`.

❓ **გაურკვეველია:** —

➡️ **რა ვკითხოთ IT-ს:** —

დამატებითი რჩევები შეხვედრისთვის

სიტუაცია	რას ვაკეთებთ
კითხვაზე პასუხი არ ვიცი	„კარგი კითხვაა, ზუსტად არ ვიცი. ჩავიწერ და დღესვე/ხვალ მოგწერთ.“ — და ნამდვილად ჩავიწერ
IT უარს ამბობს რამეზე	„რა იქნებოდა თქვენთვის მისაღები ალტერნატივა?“ — დოკუმენტ 04-ში თითო მოთხოვნას ალტერნატივა აქვს
ტექნიკური დეტალი მაინტერესებს, მაგრამ დრო არ არის	„ამას ცალკე ტექნიკურ შეხვედრაზე ან მეილით გავაგრძელოთ?“
გვეკითხებიან ციფრს, რომელიც არ გაგვიზომავს	„ეს შეფასებაა, არა გაზომვა. პილოტზე გავზომავთ.“
საუბარი უსაფრთხოებაზე გადადის	კრიტიკას ვეთანხმებით: „ეს ჩვენს გასაკეთებელ სიაშია (G...)“ — და თავს არ ვიმართლებთ
გვეკითხებიან „რატომ ასე და არა ისე“	ვხსნით მიზეზს და ვამატებთ: „თქვენი სტანდარტი თუ სხვაა, მოვერგებით“

გენერირებულია 03-IT-შეხვედრის-კითხვები-და-პასუხები.md-იდან — სწორდება მხოლოდ .md ფაილი.